

**ĐẠI HỌC QUỐC GIA HÀ NỘI**  
**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**  
**KHOA VẬT LÝ**



**NGUYỄN TIẾN DŨNG**

**NGHIÊN CỨU HIỆU QUẢ CÁC KIẾN TRÚC HỌC**  
**SÂU TRONG BÀI TOÁN NHẬN DIỆN VĂN BẢN**  
**TIẾNG VIỆT**

Đồ án Kỹ thuật Điện tử và Tin học

Ngành Kỹ thuật Điện tử và Tin học

(Chương trình đào tạo chuẩn)

**Hà Nội - 2026**

**ĐẠI HỌC QUỐC GIA HÀ NỘI**  
**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**  
**KHOA VẬT LÝ**



**NGUYỄN TIẾN DŨNG**

**NGHIÊN CỨU HIỆU QUẢ CÁC KIẾN TRÚC HỌC**  
**SÂU TRONG BÀI TOÁN NHẬN DIỆN VĂN BẢN**  
**TIẾNG VIỆT**

Đồ án Kỹ thuật Điện tử và Tin học

Ngành Kỹ thuật Điện tử và Tin học

(Chương trình đào tạo chuẩn)

**Giảng viên hướng dẫn : CN. Vi Anh Quân**

**Hà Nội - 2026**

## ***Lời cảm ơn***

Trong quá trình thực hiện môn học Đồ án Kỹ thuật Điện tử và Tin học để hoàn thành bài báo cáo này, em đã nhận được rất nhiều sự quan tâm, hướng dẫn và hỗ trợ quý báu từ thầy Vi Anh Quân. Em xin bày tỏ lòng biết ơn chân thành và sâu sắc đến thầy vì sự tận tâm trong giảng dạy, những góp ý chuyên môn quý báu cũng như sự hỗ trợ nhiệt tình trong suốt quá trình học tập và thực hiện đề tài.

Những kiến thức và kinh nghiệm mà thầy truyền đạt không chỉ giúp em hoàn thành tốt bài báo cáo mà còn là nền tảng quan trọng cho quá trình học tập và phát triển trong tương lai. Sự nghiêm túc, nhiệt huyết và tinh thần trách nhiệm của thầy đã trở thành động lực lớn để em nỗ lực hơn trong quá trình nghiên cứu và thực hiện đề tài.

Mặc dù đã cố gắng hoàn thiện đề tài này một cách tốt nhất, do kiến thức và kinh nghiệm thực tiễn còn hạn chế nên khó tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những ý kiến đóng góp và nhận xét từ thầy để có thể tiếp tục hoàn thiện và nâng cao kiến thức của bản thân.

Lời cuối cùng, em xin kính chúc thầy luôn mạnh khỏe, hạnh phúc và đạt nhiều thành công trong sự nghiệp giảng dạy của mình.

Em xin chân thành cảm ơn!

# Mục lục

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>Lời cảm ơn</b>                                                    | <b>i</b>  |
| <b>Danh sách hình vẽ</b>                                             | <b>iv</b> |
| <b>Danh sách bảng</b>                                                | <b>v</b>  |
| <b>Danh sách tên viết tắt</b>                                        | <b>vi</b> |
| <b>MỞ ĐẦU</b>                                                        | <b>1</b>  |
| <b>1 GIỚI THIỆU</b>                                                  | <b>2</b>  |
| 1.1 Đặt vấn đề . . . . .                                             | 2         |
| 1.2 Mục tiêu nghiên cứu . . . . .                                    | 2         |
| <b>2 TỔNG QUAN</b>                                                   | <b>4</b>  |
| 2.1 Tổng quan về bài toán nhận diện chữ viết tay . . . . .           | 4         |
| 2.2 Các thách thức trong nhận diện chữ viết tay tiếng việt . . . . . | 4         |
| 2.2.1 Một số thách thức của tiếng Việt . . . . .                     | 4         |
| 2.2.2 Sự khác nhau giữa chữ đánh máy và chữ viết tay . . . . .       | 6         |
| 2.3 Nghiên cứu liên quan . . . . .                                   | 6         |
| 2.3.1 Phương pháp truyền thống . . . . .                             | 6         |
| 2.3.2 Phương pháp hiện đại . . . . .                                 | 7         |
| 2.3.3 Phương pháp đề xuất . . . . .                                  | 8         |
| 2.4 Tổng quan phương pháp . . . . .                                  | 10        |
| 2.5 Mô hình YOLO . . . . .                                           | 11        |
| 2.5.1 Giới thiệu . . . . .                                           | 11        |
| 2.5.2 YOLO26 . . . . .                                               | 12        |
| 2.6 Mạng CRNN . . . . .                                              | 15        |
| 2.6.1 Mạng CNN . . . . .                                             | 16        |
| 2.6.2 Mạng RNN . . . . .                                             | 18        |
| 2.6.3 Mạng BiLSTM . . . . .                                          | 21        |
| <b>3 PHƯƠNG PHÁP</b>                                                 | <b>23</b> |
| 3.1 Module 1 : Phát hiện dòng chữ trong văn bản . . . . .            | 23        |
| 3.1.1 Chuẩn bị dữ liệu . . . . .                                     | 24        |
| 3.1.2 Tiền xử lý dữ liệu . . . . .                                   | 25        |
| 3.1.3 Hàm mất mát của YOLO26 . . . . .                               | 26        |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 3.2      | Module 2: Nhận diện văn bản chữ viết tay . . . . . | 28        |
| 3.2.1    | Tiền xử lý ảnh . . . . .                           | 30        |
| 3.2.2    | Quá trình làm dữ liệu . . . . .                    | 32        |
| 3.2.3    | Dataset . . . . .                                  | 33        |
| 3.2.4    | Hàm mất mát . . . . .                              | 34        |
| <b>4</b> | <b>Thực nghiệm và thảo luận</b>                    | <b>36</b> |
| 4.1      | Kết quả thực nghiệm . . . . .                      | 36        |
| 4.1.1    | Đánh giá mô hình . . . . .                         | 36        |
| 4.1.2    | Phân tích và đánh giá kết quả . . . . .            | 37        |
| <b>5</b> | <b>Tổng kết và hướng phát triển</b>                | <b>39</b> |
| 5.1      | Tổng kết . . . . .                                 | 39        |
| 5.2      | Hạn chế và hướng phát triển . . . . .              | 39        |
|          | <b>Tài liệu tham khảo</b>                          | <b>41</b> |
| <b>A</b> | <b>Các chỉ số đánh giá</b>                         | <b>44</b> |

## Danh sách hình vẽ

|      |                                                                   |    |
|------|-------------------------------------------------------------------|----|
| 2.1  | Hình ảnh về vấn đề của tiếng việt . . . . .                       | 5  |
| 2.2  | Hình ảnh phương pháp Segmentation-based . . . . .                 | 7  |
| 2.3  | Hình ảnh phương pháp TROCR kết hợp SigLIP . . . . .               | 8  |
| 2.4  | Hình ảnh phương pháp CRNN [5] . . . . .                           | 9  |
| 2.5  | Hình ảnh tổng quan phương pháp . . . . .                          | 10 |
| 2.6  | Mô hình mạng CNN được sử dụng trong YOLO [15] . . . . .           | 11 |
| 2.7  | Kiến trúc tổng quan YOLO26 [18] . . . . .                         | 12 |
| 2.8  | Kiến trúc NMS-Free [18] . . . . .                                 | 13 |
| 2.9  | Loại bỏ DFL [18] . . . . .                                        | 14 |
| 2.10 | Tối ưu cho vật thể nhỏ [18] . . . . .                             | 14 |
| 2.11 | Tối ưu hóa MuSGD [18] . . . . .                                   | 15 |
| 2.12 | Mạng CRNN tổng quát [5] . . . . .                                 | 15 |
| 2.13 | Kiến trúc mạng CNN [16] . . . . .                                 | 16 |
| 2.14 | Kiến trúc mạng RNN [5] . . . . .                                  | 19 |
| 2.15 | Kiến trúc mạng BiLSTM được trải ra theo thời gian . . . . .       | 21 |
| 3.1  | Tổng quan quy trình phát hiện dòng chữ bằng YOLO26 [18] . . . . . | 23 |
| 3.2  | Hình ảnh quy trình xử lý dữ liệu . . . . .                        | 25 |
| 3.3  | Hình ảnh mạng CRNN . . . . .                                      | 28 |
| 3.4  | Quá trình xử lý dòng chữ . . . . .                                | 30 |
| 3.5  | Quá trình xử lý dữ liệu . . . . .                                 | 32 |

## Danh sách bảng

|     |                                                                                                 |    |
|-----|-------------------------------------------------------------------------------------------------|----|
| 2.1 | So sánh đặc điểm giữa chữ đánh máy và chữ viết tay . . . . .                                    | 6  |
| 3.1 | Thống kê tập dữ liệu. . . . .                                                                   | 24 |
| 3.2 | Thống kê tập dữ liệu hình ảnh dòng chữ. . . . .                                                 | 34 |
| 4.1 | Kết quả đánh giá hiệu năng của các mô hình trên tập dữ liệu chữ viết tay<br>tiếng Việt. . . . . | 36 |

## Danh sách tên viết tắt

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <b>CRNN</b>     | <b>C</b> onvolutional <b>R</b> ecurrent <b>N</b> eural <b>N</b> etwork          |
| <b>OCR</b>      | <b>O</b> ptical <b>C</b> haracter <b>R</b> ecognition                           |
| <b>CNN</b>      | <b>C</b> onvolutional <b>N</b> eural <b>N</b> etwork                            |
| <b>RNN</b>      | <b>R</b> ecurrent <b>N</b> eural <b>N</b> etwork                                |
| <b>CPU</b>      | <b>C</b> entral <b>P</b> rocessing <b>U</b> nit                                 |
| <b>BiLSTM</b>   | <b>B</b> idirectional <b>L</b> ong <b>S</b> hort- <b>T</b> erm <b>M</b> emory   |
| <b>LSTM</b>     | <b>L</b> ong <b>S</b> hort- <b>T</b> erm <b>M</b> emory                         |
| <b>ReLU</b>     | <b>R</b> ectified <b>L</b> inear <b>U</b> nit                                   |
| <b>CTC</b>      | <b>C</b> onnectionist <b>T</b> emporal <b>C</b> lassification                   |
| <b>HTR</b>      | <b>H</b> andwritten <b>T</b> ext <b>R</b> ecognition                            |
| <b>TrOCR</b>    | <b>T</b> ransformer-based <b>O</b> ptical <b>C</b> haracter <b>R</b> ecognition |
| <b>SigLIP</b>   | <b>S</b> igmoid <b>L</b> oss for <b>I</b> mage- <b>P</b> retraining             |
| <b>YOLO</b>     | <b>Y</b> ou <b>O</b> nly <b>L</b> ook <b>O</b> nce                              |
| <b>NMS</b>      | <b>N</b> on- <b>M</b> aximum <b>S</b> uppression                                |
| <b>DFL</b>      | <b>D</b> istribution <b>F</b> ocal <b>L</b> oss                                 |
| <b>STAL</b>     | <b>S</b> oft <b>T</b> arget <b>A</b> ssignment <b>L</b> oss                     |
| <b>SGD</b>      | <b>S</b> tochastic <b>G</b> radient <b>D</b> escent                             |
| <b>ProgLoss</b> | <b>P</b> rogressive <b>L</b> oss <b>B</b> alancing                              |



## MỞ ĐẦU

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ cùng với sự phát triển vượt bậc của công nghệ thông tin, nhu cầu số hóa tài liệu để lưu trữ, quản lý và tìm kiếm ngày càng trở nên cấp thiết [1]. Tuy nhiên, trên thực tế, một lượng lớn tài liệu, đặc biệt là văn bản viết tay và văn bản tiếng Việt, vẫn chưa được số hóa hiệu quả. Các phương pháp nhập liệu thủ công không chỉ tốn nhiều thời gian, công sức mà còn dễ phát sinh sai sót, gây khó khăn trong việc khai thác và xử lý thông tin [2].

Xuất phát từ thực tế đó, em lựa chọn đề tài “Nghiên cứu hiệu quả các kiến trúc học sâu trong bài toán nhận diện văn bản tiếng Việt”. Đề tài tập trung vào việc nhận diện chữ viết tay tiếng Việt và khảo sát, xây dựng và so sánh hiệu quả của các mô hình học sâu tiêu biểu trong lĩnh vực nhận diện văn bản, nhằm tìm ra hướng tiếp cận phù hợp cho dữ liệu tiếng Việt – một ngôn ngữ có đặc thù về dấu và cấu trúc ký tự phức tạp [3].

Trong báo cáo này, em trình bày quá trình nghiên cứu, thiết kế mô hình, huấn luyện và đánh giá các kiến trúc học sâu khác nhau áp dụng cho bài toán nhận diện văn bản chữ viết tay. Qua đó, đề tài hướng tới việc đưa ra những nhận xét, so sánh và đề xuất giải pháp nhằm nâng cao hiệu quả nhận diện, góp phần cải thiện chất lượng các hệ thống số hóa tài liệu tiếng Việt trong thực tế.

# Chương 1 GIỚI THIỆU

## 1.1 Đặt vấn đề

Mặc dù công nghệ xử lý văn bản đã có nhiều bước tiến đáng kể, việc nhận diện và số hóa văn bản tiếng Việt, đặc biệt là chữ viết tay, vẫn là một bài toán chưa được giải quyết triệt để [3]. Những hạn chế trong việc xử lý tự động các loại dữ liệu này đang ảnh hưởng trực tiếp đến hiệu quả lưu trữ và khai thác thông tin trong thực tế [3].

Bài toán nhận diện văn bản, mặc dù đã đạt được nhiều tiến bộ đáng kể nhờ sự phát triển của trí tuệ nhân tạo và học sâu, vẫn còn tồn tại nhiều thách thức khi áp dụng cho tiếng Việt. Đặc thù ngôn ngữ với hệ thống dấu phong phú, cùng với sự đa dạng trong phong cách chữ viết tay, làm gia tăng độ phức tạp của bài toán và ảnh hưởng trực tiếp đến độ chính xác của các mô hình nhận diện. Bên cạnh đó, hiệu quả của các mô hình học sâu phụ thuộc lớn vào kiến trúc được sử dụng, trong khi việc đánh giá và so sánh một cách hệ thống giữa các kiến trúc này đối với dữ liệu tiếng Việt vẫn còn hạn chế [4].

Việc nghiên cứu và đánh giá hiệu quả của các kiến trúc học sâu trong bài toán nhận diện văn bản tiếng Việt là cần thiết. Thông qua việc phân tích, xây dựng và so sánh các mô hình khác nhau, có thể xác định được những phương pháp phù hợp, góp phần nâng cao độ chính xác và tính ổn định của phương pháp nhận diện, đồng thời đáp ứng tốt hơn nhu cầu số hóa tài liệu trong thực tế.

## 1.2 Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là nghiên cứu phương pháp nhận dạng chữ viết tay tiếng Việt dựa trên mô hình CRNN (Convolutional Recurrent Neural Network), từ đó phân tích khả năng ứng dụng của mô hình trong bài toán OCR chữ viết tay [5]. Đề tài tập trung khai thác khả năng kết hợp giữa mạng tích chập CNN trong việc trích xuất đặc

trưng ảnh và mạng hồi tiếp RNN/BiLSTM trong việc xử lý dữ liệu tuần tự để nhận dạng chuỗi ký tự tiếng Việt một cách hiệu quả [6].

Bên cạnh đó, đề tài cũng triển khai và đánh giá thêm một số phương pháp nhận dạng khác nhằm so sánh hiệu quả với mô hình CRNN, từ đó đưa ra nhận xét về độ chính xác và khả năng phù hợp của từng phương pháp đối với dữ liệu chữ viết tay tiếng Việt.

Để đạt được mục tiêu trên, đề tài tập trung thực hiện các nội dung sau:

Nghiên cứu mô hình CRNN trong bài toán OCR: Tìm hiểu cấu trúc và nguyên lý hoạt động của mô hình CRNN, bao gồm quá trình trích xuất đặc trưng bằng CNN, xử lý chuỗi bằng RNN/BiLSTM và giải mã ký tự bằng hàm mất mát CTC (Connectionist Temporal Classification) [7].

Nghiên cứu phương pháp xử lý dữ liệu ảnh chữ viết tay: Khảo sát các kỹ thuật tiền xử lý ảnh như chuyển ảnh xám, khử nhiễu, nhị phân hóa và chuẩn hóa kích thước nhằm cải thiện chất lượng dữ liệu đầu vào cho mô hình nhận dạng.

So sánh với các phương pháp nhận dạng khác: Triển khai thêm một số mô hình hoặc phương pháp OCR khác để tiến hành đánh giá và so sánh với CRNN dựa trên các tiêu chí như độ chính xác nhận dạng, khả năng xử lý ký tự tiếng Việt và tốc độ thực thi. Phân tích kết quả thu được từ quá trình huấn luyện và kiểm thử nhằm đánh giá hiệu quả của các mô hình trong bài toán nhận dạng chữ viết tay tiếng Việt [5], [7].

## Chương 2 TỔNG QUAN

### 2.1 Tổng quan về bài toán nhận diện chữ viết tay

Nhận diện chữ viết tay (Handwritten Text Recognition - HTR) là một bài toán quan trọng trong lĩnh vực xử lý ảnh và thị giác máy tính, với mục tiêu chuyển đổi nội dung chữ viết tay từ ảnh sang dạng văn bản có thể xử lý được bằng máy tính [5], [8]. Bài toán này đóng vai trò quan trọng trong việc số hóa tài liệu, tự động hóa quy trình nhập liệu và lưu trữ thông tin.

HTR được chia thành hai loại chính: nhận diện chữ viết tay trực tuyến (online) và ngoại tuyến (offline). Trong đó, bài toán offline HTR chỉ sử dụng ảnh tĩnh của chữ viết tay, không có thông tin về quá trình viết, do đó khó hơn và phổ biến hơn trong thực tế [8].

Ngoài ra, bài toán HTR có thể được chia theo mức độ xử lý, bao gồm nhận diện ở mức ký tự (character-level), từ (word-level) và dòng (line-level). Trong đề tài này, bài toán được tập trung ở mức dòng chữ (line-level), trong đó mỗi ảnh đầu vào tương ứng với một chuỗi ký tự cần được nhận diện.

### 2.2 Các thách thức trong nhận diện chữ viết tay tiếng việt

Mặc dù lĩnh vực nhận diện chữ viết tay tiếng việt đã đạt được nhiều tiến bộ nhờ sự phát triển của các phương pháp học sâu, đây vẫn là một bài toán thách thức do tính không chuẩn hóa và biến thiên cao của dữ liệu đầu vào.

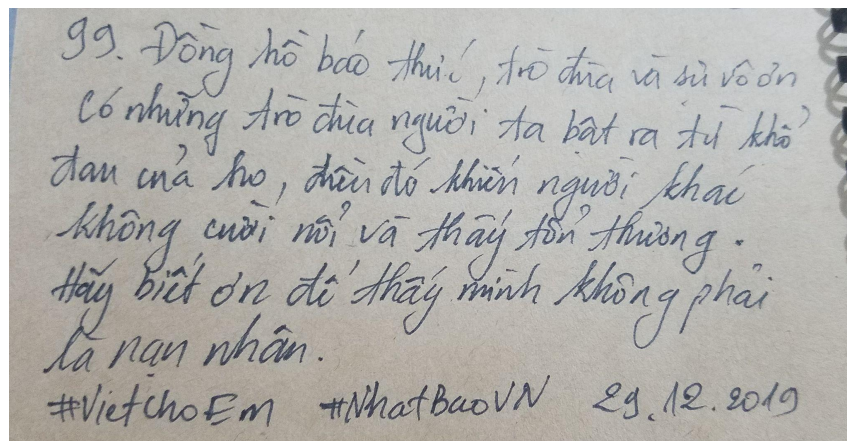
#### 2.2.1 Một số thách thức của tiếng Việt

Trước hết, sự đa dạng trong phong cách chữ viết khiến các ký tự cùng một lớp có thể khác nhau đáng kể về hình dạng, độ nghiêng, độ đậm nhạt và cách thể hiện nét, gây khó

khăn cho việc học đặc trưng chung của mô hình. Bên cạnh đó, hiện tượng các ký tự dính liền hoặc chồng lấn trong chữ viết tay tự nhiên làm cho việc phân đoạn trở nên phức tạp, đặc biệt đối với các phương pháp truyền thống [9], [10].

Ngoài ra, chất lượng dữ liệu đầu vào không đồng đều, chịu ảnh hưởng bởi nhiễu, điều kiện ánh sáng và phương thức thu thập, cũng làm giảm hiệu quả nhận diện [11]. Các biến dạng hình học như chữ nghiêng, cong hoặc lệch dòng tiếp tục làm gia tăng độ khó của bài toán.

Đặc biệt, đối với tiếng Việt, hệ thống dấu thanh và dấu phụ phong phú làm tăng đáng kể độ phức tạp, do các dấu này thường nhỏ, dễ bị nhiễu hoặc dính với ký tự chính, từ đó ảnh hưởng đến độ chính xác của phương pháp nhận diện.



Hình 2.1: Hình ảnh về vấn đề của tiếng việt

### 2.2.2 Sự khác nhau giữa chữ đánh máy và chữ viết tay

| Tiêu chí          | Chữ đánh máy       | Chữ viết tay                |
|-------------------|--------------------|-----------------------------|
| Chuẩn hóa ký tự   | Cố định theo font  | Phụ thuộc người viết        |
| Độ đồng nhất      | Gần như giống nhau | Biến thiên lớn giữa các mẫu |
| Hình dạng ký tự   | Rõ ràng, ổn định   | Méo, thiếu hoặc thừa nét    |
| Khoảng cách ký tự | Cách đều nhau      | Không đều, dễ dính chữ      |
| Dòng chữ          | Thẳng hàng         | Có thể nghiêng, cong        |
| Nhiều dữ liệu     | Thấp               | Cao (mờ, rung, ánh sáng)    |
| Khó khăn OCR      | Dễ xử lý           | Khó, dễ nhầm ký tự          |

Bảng 2.1: So sánh đặc điểm giữa chữ đánh máy và chữ viết tay

## 2.3 Nghiên cứu liên quan

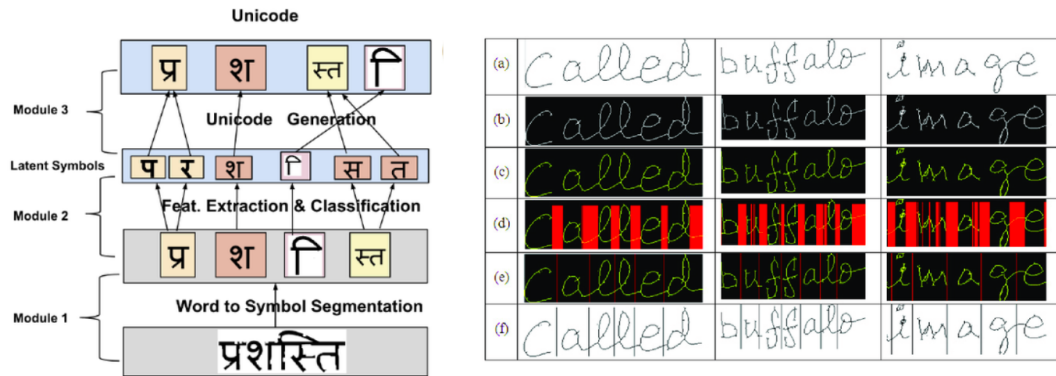
Bài toán nhận diện chữ viết tay (Handwritten Text Recognition - HTR) đã được nghiên cứu từ lâu và trải qua nhiều giai đoạn phát triển, từ các phương pháp truyền thống dựa trên đặc trưng thủ công đến các phương pháp hiện đại dựa trên học sâu.

### 2.3.1 Phương pháp truyền thống

Phương pháp OCR dựa trên segmentation là hướng tiếp cận truyền thống, trong đó bài toán được chia thành nhiều bước nhỏ, đặc biệt là bước tách (phân đoạn) ký tự [2]. Hệ thống sẽ lần lượt thực hiện: tiền xử lý ảnh, phân đoạn dòng chữ, tách từ và cuối cùng là tách từng ký tự riêng lẻ trước khi đưa vào mô hình nhận diện.

Sau khi ký tự được tách ra, mỗi ký tự sẽ được đưa vào bộ phân loại (như SVM, k-NN hoặc mạng neural đơn giản) để nhận diện.

Ưu điểm của phương pháp này là dễ hiểu và phù hợp với văn bản in rõ ràng, có cấu trúc ổn định. Tuy nhiên, nó gặp nhiều hạn chế khi áp dụng cho chữ viết tay do hiện tượng ký tự dính liền, biến dạng hoặc không có ranh giới rõ ràng, dẫn đến sai sót trong bước phân đoạn và kéo theo giảm độ chính xác [2].



Hình 2.2: Hình ảnh phương pháp Segmentation-based

### 2.3.2 Phương pháp hiện đại

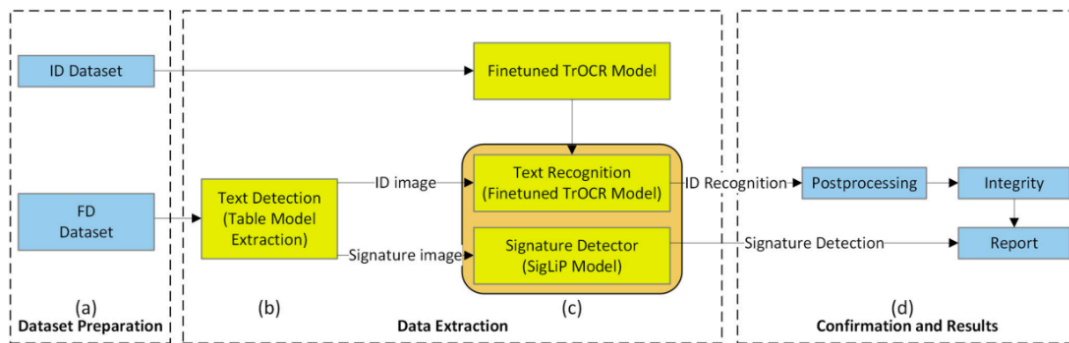
Phương pháp nhận diện chữ viết tay dựa trên Transformer là hướng tiếp cận hiện đại, trong đó bài toán được xây dựng theo mô hình end-to-end mà không cần thực hiện bước phân đoạn ký tự như các phương pháp truyền thống. Một trong những mô hình tiêu biểu là TrOCR, thường được kết hợp với mô hình thị giác mạnh như SigLIP để nâng cao khả năng trích xuất đặc trưng [12], [13].

Hệ thống sẽ trực tiếp nhận ảnh đầu vào và thực hiện học ánh xạ từ hình ảnh sang chuỗi văn bản thông qua kiến trúc Transformer [14]. Cụ thể, ảnh được đưa vào encoder để trích xuất đặc trưng toàn cục, sau đó decoder sẽ sinh ra chuỗi ký tự tương ứng theo từng bước, sử dụng cơ chế attention để tập trung vào các vùng quan trọng của ảnh.

Khi kết hợp với SigLIP, mô hình có khả năng học biểu diễn hình ảnh gắn với ngữ nghĩa tốt hơn, giúp cải thiện hiệu quả trong các trường hợp chữ viết tay phức tạp, nhiễu hoặc có sự đa dạng lớn về phong cách. Ngoài ra, SigLIP còn hỗ trợ các tác vụ như phát hiện hoặc xác thực chữ ký thông qua việc so sánh embedding [13].

Ưu điểm của phương pháp này là không cần tách ký tự riêng lẻ, giảm thiểu sai số do phân đoạn, đồng thời có khả năng mô hình hóa ngữ cảnh chuỗi tốt hơn nhờ cơ chế attention. Bên cạnh đó, mô hình có thể tận dụng các kỹ thuật tiền huấn luyện trên tập dữ liệu lớn để đạt hiệu năng cao.

Tuy nhiên, phương pháp này cũng tồn tại một số hạn chế như yêu cầu tài nguyên tính toán lớn, thời gian huấn luyện dài và phụ thuộc nhiều vào dữ liệu huấn luyện, ngoài ra việc triển khai và tối ưu mô hình cũng phức tạp.



Hình 2.3: Hình ảnh phương pháp TROCR kết hợp SigLIP

### 2.3.3 Phương pháp đề xuất

Trong bối cảnh các phương pháp hiện đại như Transformer đạt hiệu năng cao nhưng đòi hỏi tài nguyên lớn và dữ liệu huấn luyện quy mô lớn, bài toán nhận diện chữ viết tay tiếng Việt vẫn cần một giải pháp cân bằng giữa hiệu quả và tính khả thi. Do đó, trong nghiên cứu này, em đề xuất sử dụng mô hình CRNN kết hợp CTC Loss như một hướng tiếp cận phù hợp và hiệu quả [5], [7].

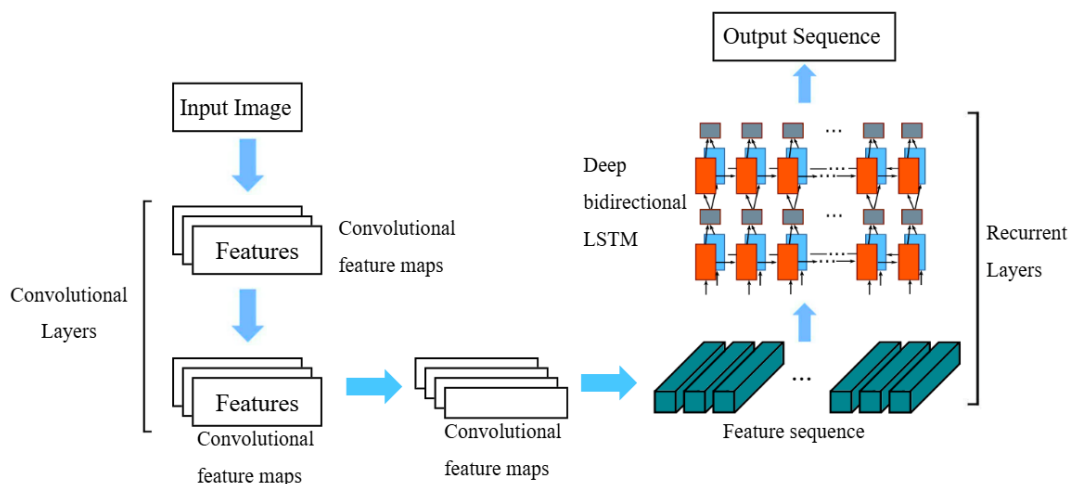
Phương pháp này được xây dựng theo hướng end-to-end, cho phép nhận diện trực tiếp từ ảnh đầu vào sang chuỗi ký tự đầu ra mà không cần thực hiện bước phân đoạn ký tự riêng lẻ. Cách tiếp cận này giúp đơn giản hóa quy trình xử lý và nâng cao độ chính xác tổng thể [5], [6], [7].



Ảnh đầu vào trước tiên được đưa qua các lớp convolution để trích xuất đặc trưng quan trọng như nét chữ, hình dạng và cấu trúc ký tự. Sau đó, đặc trưng này được chuyển đổi thành chuỗi theo chiều ngang và đưa vào mạng BiLSTM nhằm học mối quan hệ phụ thuộc giữa các ký tự trong từ hoặc câu [6]. Cuối cùng, CTC Loss được sử dụng để tính toán sai số mà không cần căn chỉnh trực tiếp giữa đầu ra của mô hình và nhãn, giúp đơn giản hóa quá trình huấn luyện [7].

Ưu điểm của phương pháp CRNN + CTC là kiến trúc đơn giản, dễ triển khai, không yêu cầu dữ liệu gán nhãn chi tiết theo từng ký tự và có thể huấn luyện trên các tập dữ liệu vừa và nhỏ. Ngoài ra, mô hình có tốc độ suy luận nhanh, phù hợp với các hệ thống thực tế.

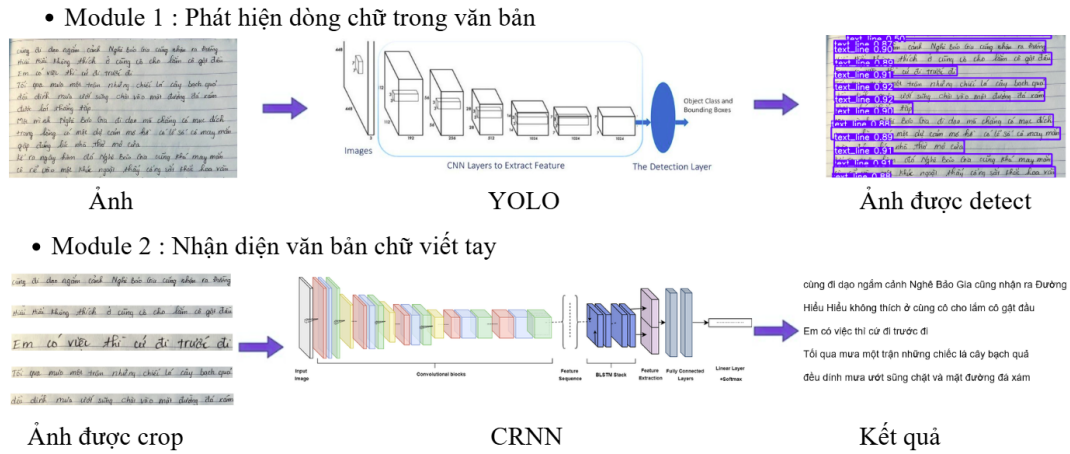
Tuy nhiên, phương pháp này cũng tồn tại một số hạn chế như khả năng mô hình hóa ngữ cảnh dài hạn chưa mạnh bằng các mô hình Transformer, và hiệu năng có thể bị ảnh hưởng khi dữ liệu quá phức tạp hoặc nhiễu loạn. Dù vậy, với sự cân bằng giữa hiệu năng và chi phí tính toán, CRNN + CTC vẫn là một lựa chọn mạnh mẽ và thực tiễn cho bài toán nhận diện chữ viết tay tiếng Việt [5].



Hình 2.4: Hình ảnh phương pháp CRNN [5]

## 2.4 Tổng quan phương pháp

Phương pháp nhận diện chữ viết tay tiếng Việt trong nghiên cứu này được thiết kế theo hướng tiếp cận hai giai đoạn (two-stage pipeline), bao gồm :



Hình 2.5: Hình ảnh tổng quan phương pháp

Ở giai đoạn đầu tiên, hệ thống sử dụng mô hình phát hiện đối tượng YOLO để xác định vị trí các dòng chữ trong ảnh đầu vào [15]. Cụ thể, ảnh tài liệu viết tay ban đầu sẽ được đưa qua mạng CNN để trích xuất đặc trưng, sau đó thông qua lớp detection để dự đoán các bounding box tương ứng với từng dòng văn bản. Kết quả của bước này là tập hợp các vùng ảnh chứa từng dòng chữ riêng biệt .

Trong giai đoạn thứ hai, mỗi dòng văn bản sau khi được cắt (crop) sẽ được đưa vào mô hình nhận diện chữ viết tay dựa trên kiến trúc CRNN (Convolutional Recurrent Neural Network) [5]. Mô hình này kết hợp :

- Mạng CNN dùng để trích xuất đặc trưng không gian từ ảnh đầu vào [16].
- Mạng RNN hai chiều (BiLSTM) dùng để mô hình hóa chuỗi và khai thác thông tin ngữ cảnh [17].
- Hàm mất mát CTC dùng để thực hiện ánh xạ giữa chuỗi đặc trưng và nhãn đầu ra mà không cần căn chỉnh thủ công [7].

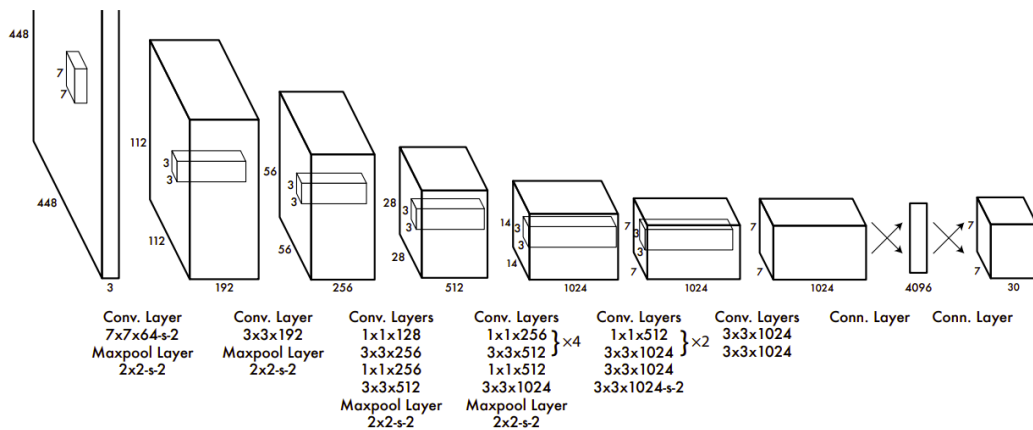
Đầu ra cuối cùng của phương pháp là nội dung văn bản được nhận diện tương ứng với từng dòng chữ trong ảnh ban đầu.

## 2.5 Mô hình YOLO

### 2.5.1 Giới thiệu

YOLO là 1 từ viết tắt cho ‘You only look once’ (Bạn chỉ nhìn một lần) là một thuật toán phát hiện đối tượng chia hình ảnh thành một hệ thống lưới. Mỗi ô trong lưới chịu trách nhiệm phát hiện các đối tượng trong chính nó. YOLO là một trong những thuật toán phát hiện đối tượng nổi tiếng nhất do tốc độ và độ chính xác của nó [15].

YOLO là một mô hình mạng CNN cho việc phát hiện, nhận dạng, phân loại đối tượng. YOLO được tạo ra từ việc kết hợp giữa các convolutional layers và connected layers [15], [16]. Trong đó các convolutional layers sẽ trích xuất ra các feature của ảnh, còn fullconnected layers sẽ dự đoán ra xác suất đó và tọa độ của đối tượng.



Hình 2.6: Mô hình mạng CNN được sử dụng trong YOLO [15]

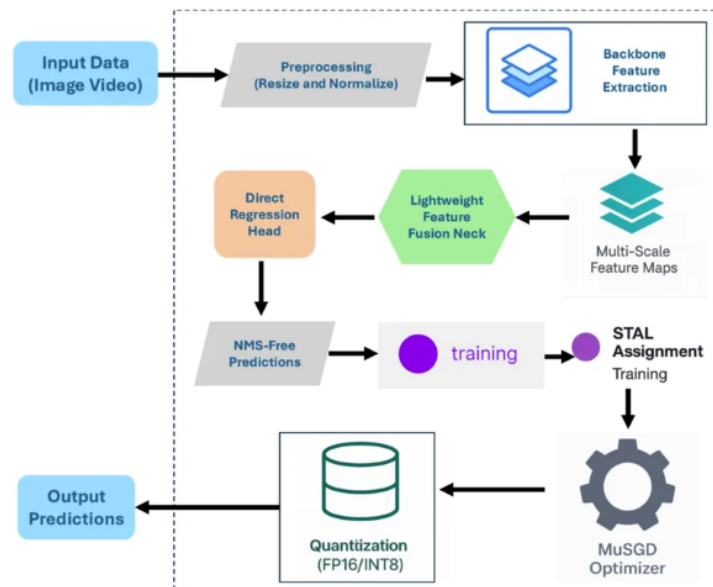
Hình ảnh đầu vào sẽ được chia thành một lưới kích thước  $S \times S$ , trong đó mỗi ô lưới chịu trách nhiệm phát hiện các đối tượng có tâm nằm trong ô đó. Đối với mỗi ô, mô hình sẽ dự đoán một số lượng bounding box cùng với thông tin về vị trí và xác suất lớp.

Với mỗi bounding box, mô hình dự đoán 5 tham số bao gồm  $(x, y, w, h, confidence)$ . Trong đó,  $(x, y)$  là tọa độ tâm của bounding box tương đối so với ô lưới, còn  $(w, h)$  là chiều rộng và chiều cao được chuẩn hóa theo kích thước toàn ảnh. Giá trị *confidence* thể hiện mức độ tin cậy của bounding box, được tính dựa trên xác suất tồn tại đối tượng và độ chồng lấp giữa bounding box dự đoán và ground truth.

Ngoài ra, mỗi ô lưới còn dự đoán xác suất thuộc về các lớp đối tượng dưới dạng  $Pr(Class_i | Object)$ . Xác suất cuối cùng của một lớp được tính bằng cách kết hợp giữa xác suất lớp và *confidence* của bounding box.

### 2.5.2 YOLO26

YOLO26 (hay YOLOv26) là thế hệ mới nhất thuộc họ mô hình phát hiện đối tượng You Only Look Once (YOLO) do Ultralytics phát triển, được giới thiệu vào khoảng cuối năm 2025 [18], [19]. Khác với các phiên bản trước chủ yếu tập trung vào việc gia tăng độ chính xác thông qua việc mở rộng kích thước mô hình, YOLO26 được thiết kế lại nhằm tối ưu hóa cho việc triển khai trên các thiết bị biên (edge devices), hệ thống nhúng và môi trường có tài nguyên hạn chế.

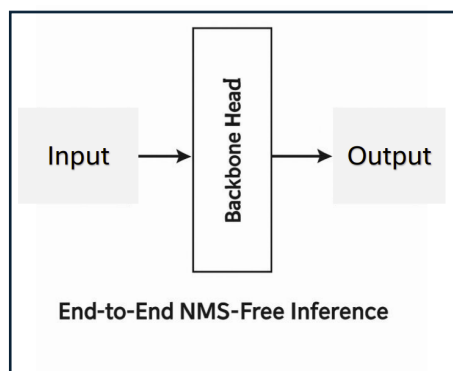


Hình 2.7: Kiến trúc tổng quan YOLO26 [18]

## 1. Kiến trúc NMS-Free

Các phiên bản YOLO trước đây (như YOLOv8, YOLOv9) yêu cầu bước hậu xử lý Non-Maximum Suppression (NMS) để loại bỏ các bounding box trùng lặp. Tuy nhiên, YOLO26 được thiết kế theo hướng end-to-end hoàn chỉnh, cho phép mô hình trực tiếp xuất ra các dự đoán cuối cùng mà không cần áp dụng NMS [18].

Việc loại bỏ bước NMS giúp giảm độ trễ của toàn hệ thống và đơn giản hóa pipeline suy luận, đặc biệt phù hợp với các ứng dụng thời gian thực.

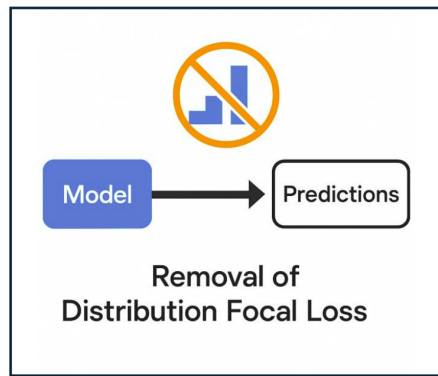


Hình 2.8: Kiến trúc NMS-Free [18]

## 2. Loại bỏ Distribution Focal Loss

Trong các phiên bản trước, Distribution Focal Loss (DFL) được sử dụng để cải thiện độ chính xác của bounding box. Tuy nhiên, DFL gây khó khăn trong việc chuyển đổi mô hình sang các định dạng triển khai như TensorRT, ONNX hay CoreML.

YOLO26 loại bỏ hoàn toàn DFL, giúp quá trình export và triển khai mô hình trở nên đơn giản, ổn định và tương thích tốt hơn với phần cứng [18].

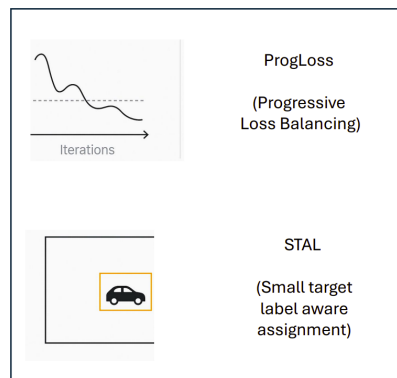


Hình 2.9: Loại bỏ DFL [18]

### 3. Tối ưu phát hiện vật thể nhỏ

YOLO26 áp dụng cơ chế gán nhãn mới mang tên Small-Target-Aware Label Assignment (STAL), kết hợp với hàm mất mát cải tiến. Cơ chế này giúp mô hình nhạy hơn với các vật thể nhỏ hoặc ở khoảng cách xa[18], [19].

Nhờ đó, hiệu năng phát hiện các đối tượng nhỏ như biển số xe hoặc chi tiết chữ viết tay được cải thiện đáng kể so với các phiên bản trước.

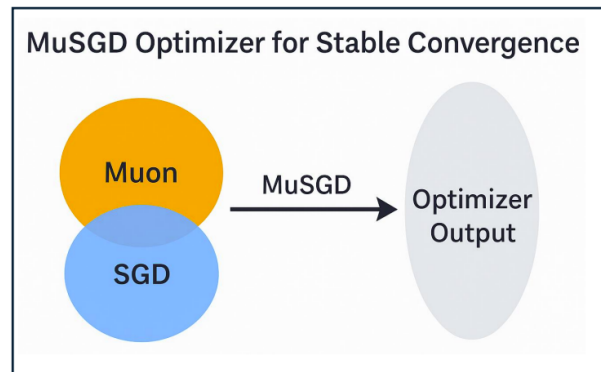


Hình 2.10: Tối ưu cho vật thể nhỏ [18]

### 4. Trình tối ưu hóa MuSGD

YOLO26 sử dụng trình tối ưu hóa MuSGD, kết hợp giữa Stochastic Gradient Descent (SGD) và thuật toán Muon. Phương pháp này giúp cải thiện tốc độ hội tụ và đảm bảo sự ổn định trong quá trình huấn luyện[18].

Việc áp dụng MuSGD cho phép mô hình đạt hiệu năng cao mà vẫn duy trì độ ổn định khi huấn luyện trên các tập dữ liệu lớn.

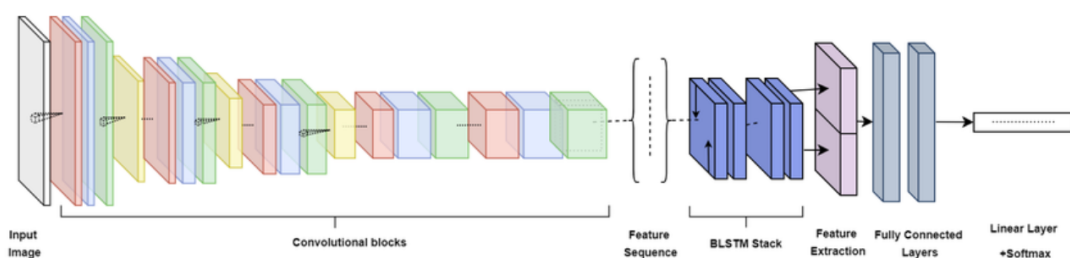


Hình 2.11: Tối ưu hóa MuSGD [18]

Nhờ đó, YOLO26 có thể được sử dụng như một nền tảng thống nhất cho nhiều ứng dụng khác nhau trong thị giác máy tính.

## 2.6 Mạng CRNN

Tên đầy đủ của CRNN là *Convolutional Recurrent Neural Network*, là một kiến trúc mạng nơ-ron kết hợp những ưu điểm của mạng nơ-ron tích chập (CNN) và mạng nơ-ron hồi quy (RNN) [5], [16], [17].



Hình 2.12: Mạng CRNN tổng quát [5]

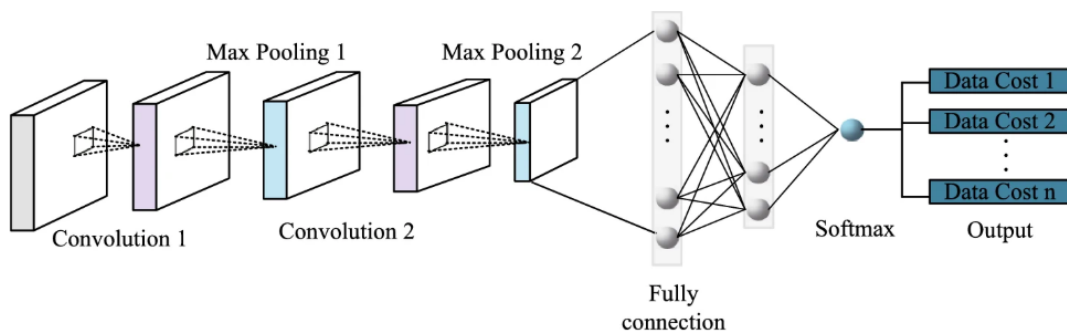
Mạng nơ-ron hồi quy tích chập (CRNN) thường được sử dụng để xử lý và phân loại dữ liệu chuỗi như giọng nói, văn bản và hình ảnh. Khả năng xử lý dữ liệu chuỗi có độ dài thay đổi và nắm bắt các mối quan hệ phụ thuộc dài hạn khiến mô hình đặc biệt hiệu

quả trong các nhiệm vụ yêu cầu hiểu và mô hình hóa thông tin ngữ cảnh và thời gian. CRNN là một công cụ mạnh mẽ để mô hình hóa và xử lý dữ liệu chuỗi, thể hiện hiệu suất tốt trong nhiều bài toán khác nhau [5].

### 2.6.1 Mạng CNN

Kiến trúc mạng nơ-ron tích chập (CNN) trong sơ đồ được xây dựng theo mô hình học sâu điển hình, bao gồm hai thành phần chính là khối trích xuất đặc trưng (*Feature Extraction*) và khối phân loại/quyết định (*Classification*) [16].

Hệ thống tiếp nhận ảnh đầu vào dưới dạng dữ liệu thô, sau đó xử lý tuần tự qua nhiều tầng trung gian nhằm tự động học và trích xuất các đặc trưng quan trọng của ảnh. Những đặc trưng này tiếp tục được đưa vào khối phân loại để thực hiện quá trình dự đoán và tính toán giá trị chi phí ở đầu ra, từ đó hỗ trợ mô hình tối ưu hóa độ chính xác trong quá trình huấn luyện.



Hình 2.13: Kiến trúc mạng CNN [16]

### 1. Khối trích xuất đặc trưng

Đây là giai đoạn quan trọng nhất giúp mạng học và hiểu được nội dung của hình ảnh thông qua quá trình lọc và thu gọn thông tin.

#### 1. Lớp tích chập



Các lớp tích chập sử dụng tập hợp các bộ lọc (*kernels*) có kích thước nhỏ di chuyển trên toàn bộ bề mặt ảnh đầu vào. Tại mỗi vị trí, phép tích chập giữa bộ lọc và vùng ảnh cục bộ được thực hiện nhằm tạo ra các bản đồ đặc trưng (*feature maps*).

Lớp Convolution đầu tiên thường có nhiệm vụ phát hiện các đặc trưng hình học cơ bản như cạnh, đường thẳng hoặc điểm ảnh nổi bật. Các lớp Convolution phía sau tiếp tục kết hợp những đặc trưng đơn giản này để hình thành các đặc trưng phức tạp hơn như hình dạng, họa tiết hoặc cấu trúc bề mặt.

## **2. Lớp gộp cực đại**

Được bố trí xen kẽ sau các lớp tích chập, lớp Max Pooling thực hiện việc chọn giá trị lớn nhất trong một cửa sổ trượt (ví dụ:  $2 \times 2$ ) để đại diện cho vùng dữ liệu tương ứng.

Quá trình này giúp giảm kích thước không gian của dữ liệu, từ đó giảm số lượng tham số cần huấn luyện và hạn chế hiện tượng quá khớp (*overfitting*). Đồng thời, Max Pooling còn giúp mô hình đạt được tính bất biến không gian (*spatial invariance*), cho phép mạng vẫn nhận diện được đặc trưng ngay cả khi đối tượng bị dịch chuyển nhẹ trong ảnh.

## **2. Khối kết nối đầy đủ và phân loại**

Sau khi các đặc trưng quan trọng đã được trích xuất và thu gọn, dữ liệu sẽ được đưa vào khối phân loại để thực hiện quá trình suy luận và ra quyết định.

### **1. Lớp kết nối đầy đủ**

Tại đây, các bản đồ đặc trưng nhiều chiều sẽ được trải phẳng (*flatten*) thành một vector một chiều. Mỗi neuron trong lớp Fully Connected sẽ kết nối với toàn bộ neuron của lớp trước đó. Vai trò của lớp này là thực hiện các phép kết hợp phi tuyến giữa các đặc trưng đã học nhằm đưa ra suy luận cuối cùng về dữ liệu đầu vào.

### **2. Hàm Softmax**

Softmax là hàm kích hoạt cuối cùng của mạng trước khi xuất ra kết quả dự đoán. Hàm này chuẩn hóa các giá trị đầu ra (*logits*) thành phân phối xác suất.

Mỗi giá trị đầu ra sẽ nằm trong khoảng  $[0, 1]$  và tổng tất cả các xác suất bằng 1. Điều này cho phép mô hình biểu diễn mức độ tin cậy đối với từng lớp hoặc từng giả thuyết tương ứng.

### **3. Cấu trúc đầu ra**

Khác với các mạng phân loại ảnh truyền thống thường trả về nhãn đối tượng, đầu ra của kiến trúc này được biểu diễn dưới dạng tập hợp các giá trị *Data Cost* từ 1 đến  $n$ .

Trong các bài toán xử lý tín hiệu và thị giác máy tính, *Data Cost* biểu thị mức độ sai số hoặc chi phí phạt khi gán một nhãn hay trạng thái cụ thể cho dữ liệu đầu vào. Mục tiêu của quá trình huấn luyện là cực tiểu hóa các giá trị chi phí này nhằm đạt được kết quả tối ưu.

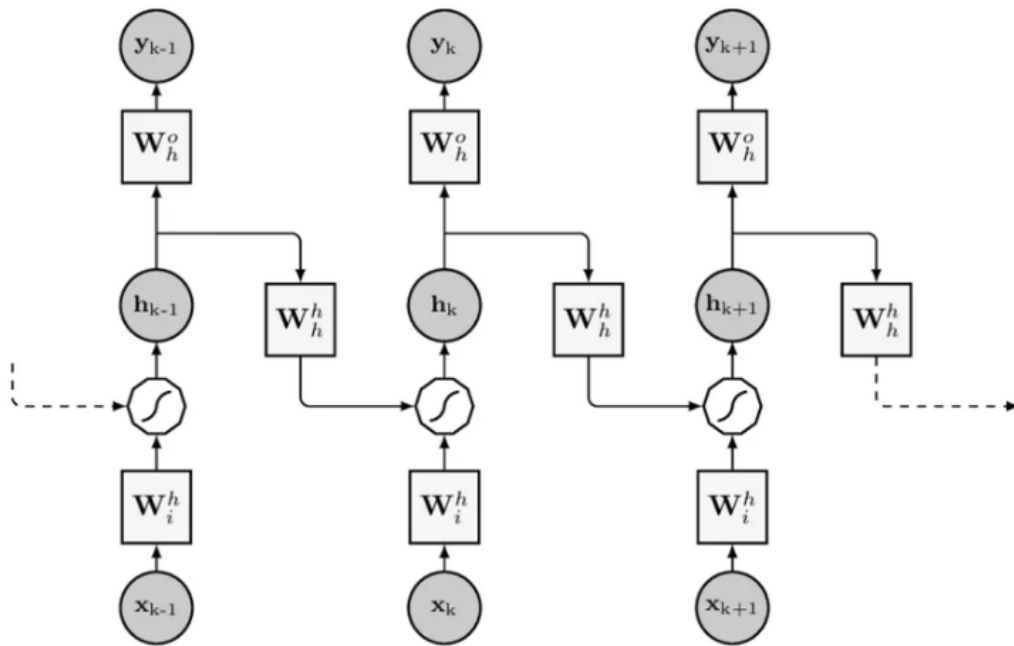
Việc mô hình xuất ra nhiều giá trị *Data Cost* cho thấy mạng đang đánh giá đồng thời nhiều giả thuyết hoặc nhiều mức độ sai lệch khác nhau. Hệ thống sau đó sẽ lựa chọn kết quả có chi phí nhỏ nhất, tương ứng với phương án có độ chính xác cao nhất.

#### **2.6.2 Mạng RNN**

Mạng nơ-ron hồi quy (RNN - Recurrent Neural Network) là một lớp (class) của mạng nơ-ron nhân tạo được thiết kế đặc biệt để xử lý dữ liệu có tính tuần tự (sequential data) hoặc dữ liệu chuỗi thời gian (time-series). Các ứng dụng phổ biến bao gồm xử lý ngôn ngữ tự nhiên (dịch máy, sinh văn bản), nhận dạng giọng nói, và dự báo chuỗi thời gian [5].

Điểm khác biệt lớn nhất giữa RNN và mạng nơ-ron truyền thẳng truyền thống (Feed-forward Neural Network) là RNN sở hữu "trí nhớ". Trong khi mạng truyền thẳng xử lý các đầu vào một cách độc lập, RNN có khả năng lưu giữ thông tin từ các bước tính toán

trước đó để gây ảnh hưởng đến kết quả của bước hiện tại. Điều này được thực hiện thông qua các kết nối hồi quy tạo thành một vòng lặp có hướng trong kiến trúc mạng.



Hình 2.14: Kiến trúc mạng RNN [5]

Hình 2.14 mô tả cấu trúc của một mạng RNN được "trải ra theo thời gian" (unrolled through time). Việc trải mạng giúp minh họa rõ quá trình luồng dữ liệu đi qua mạng tại từng bước thời gian riêng biệt (ví dụ:  $k - 1$ ,  $k$ , và  $k + 1$ ).

### Các thành phần và ký hiệu

- $\mathbf{x}_k$ : Véc-tơ dữ liệu đầu vào tại bước thời gian  $k$ .
- $\mathbf{h}_k$ : Trạng thái ẩn (hidden state) tại bước  $k$ . Thành phần này đóng vai trò là "bộ nhớ" của mạng, lưu giữ thông tin tóm tắt về chuỗi dữ liệu tính đến thời điểm hiện tại.
- $\mathbf{y}_k$ : Véc-tơ đầu ra (output) tại bước  $k$ .
- Các ma trận trọng số (Weight Matrices) cần học trong quá trình huấn luyện:
  - $\mathbf{W}_i^h$ : Trọng số kết nối từ lớp đầu vào đến trạng thái ẩn.

- $\mathbf{W}_h^h$ : Trọng số kết nối từ trạng thái ẩn của bước trước đến trạng thái ẩn của bước hiện tại. Đây là ma trận cốt lõi tạo nên vòng lặp hồi quy.
- $\mathbf{W}_h^o$ : Trọng số kết nối từ trạng thái ẩn đến lớp đầu ra.

## Nguyên lý hoạt động tại một bước thời gian $k$

Quá trình xử lý thông tin tại bước thời gian  $k$  diễn ra theo các bước sau:

### 1. Cập nhật trạng thái ẩn

Để tính toán trạng thái ẩn mới  $\mathbf{h}_k$ , mạng kết hợp dữ liệu đầu vào hiện tại  $\mathbf{x}_k$  và trạng thái ẩn từ bước ngay trước đó  $\mathbf{h}_{k-1}$ . Tổng tuyến tính của chúng được cộng thêm một véc-tơ độ lệch (bias)  $\mathbf{b}_h$  và đưa qua một hàm kích hoạt phi tuyến  $f$  (thường là tanh hoặc ReLU):

$$\mathbf{h}_k = f(\mathbf{W}_i^h \mathbf{x}_k + \mathbf{W}_h^h \mathbf{h}_{k-1} + \mathbf{b}_h)$$

### 2. Tính toán đầu ra

Sau khi trạng thái ẩn hiện tại  $\mathbf{h}_k$  được tính toán, nó được sử dụng để sinh ra đầu ra  $\mathbf{y}_k$  tương ứng. Quá trình này thực hiện bằng cách nhân  $\mathbf{h}_k$  với ma trận  $\mathbf{W}_h^o$ , cộng thêm độ lệch  $\mathbf{b}_o$  và đưa qua hàm kích hoạt đầu ra  $g$  (ví dụ như Softmax cho các bài toán phân loại):

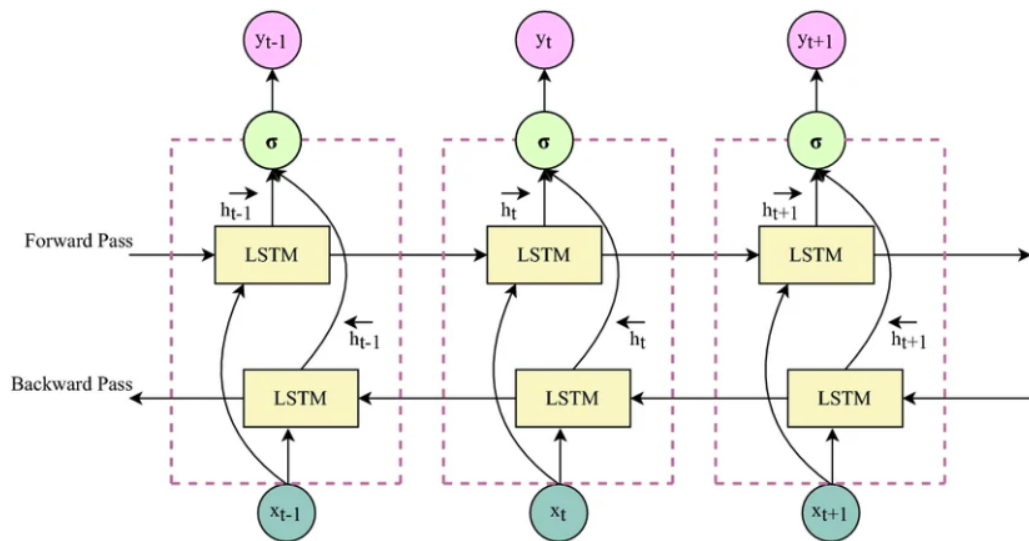
$$\mathbf{y}_k = g(\mathbf{W}_h^o \mathbf{h}_k + \mathbf{b}_o)$$

### 3. Truyền trạng thái và chia sẻ trọng số

Trạng thái ẩn  $\mathbf{h}_k$  vừa tạo ra không chỉ dùng để tính  $\mathbf{y}_k$  mà sẽ tiếp tục được truyền sang bước thời gian  $k + 1$ . Một đặc tính đặc biệt quan trọng có thể quan sát trên Hình 2.13 là các ma trận trọng số ( $\mathbf{W}_i^h$ ,  $\mathbf{W}_h^h$ ,  $\mathbf{W}_h^o$ ) được sử dụng lặp lại giống hệt nhau ở mọi bước thời gian. Việc chia sẻ trọng số này giúp giảm thiểu đáng kể số lượng tham số cần tối ưu hóa và cho phép mô hình linh hoạt xử lý các chuỗi đầu vào có độ dài bất kỳ.

### 2.6.3 Mạng BiLSTM

Mặc dù RNN hay các biến thể như LSTM truyền thống đã giải quyết tốt việc ghi nhớ thông tin từ quá khứ, chúng vẫn tồn tại một hạn chế cốt lõi: dữ liệu chỉ được xử lý theo một chiều duy nhất (từ quá khứ đến hiện tại). Tuy nhiên, trong nhiều bài toán thực tế như xử lý ngôn ngữ tự nhiên hay dịch máy, ngữ cảnh từ "tương lai" (các từ xuất hiện phía sau) đóng vai trò quan trọng không kém để hiểu trọn vẹn ý nghĩa của phần tử ở thời điểm hiện tại. Mạng bộ nhớ dài-ngắn hạn hai chiều (BiLSTM - Bidirectional LSTM) được đề xuất nhằm khắc phục triệt để nhược điểm này.



Hình 2.15: Kiến trúc mạng BiLSTM được trải ra theo thời gian

Hình 2.15 mô tả chi tiết kiến trúc của một mạng BiLSTM được trải ra qua các bước thời gian (ví dụ:  $t-1$ ,  $t$ , và  $t+1$ ). Thay vì chỉ có một chuỗi ẩn, BiLSTM duy trì hai luồng trạng thái ẩn song song:

#### 1. Các thành phần và nguyên lý hoạt động

- **Lớp truyền xuôi (Forward Pass):** Chuỗi LSTM phía trên (theo chiều mũi tên từ trái sang phải) đọc chuỗi dữ liệu đầu vào  $x$  theo thứ tự thời gian chuẩn (từ  $t = 1$  đến  $T$ ). Tại bước thời gian  $t$ , nó tính toán trạng thái ẩn xuôi  $\vec{h}_t$  dựa trên đầu vào

hiện tại  $\mathbf{x}_t$  và trạng thái ẩn của bước trước đó  $\vec{\mathbf{h}}_{t-1}$ :

$$\vec{\mathbf{h}}_t = \text{LSTM}_{forward}(\mathbf{x}_t, \vec{\mathbf{h}}_{t-1})$$

- **Lớp truyền ngược (Backward Pass):** Chuỗi LSTM phía dưới (theo chiều mũi tên từ phải sang trái) đọc chuỗi dữ liệu đầu vào  $\mathbf{x}$  theo thứ tự thời gian đảo ngược (từ  $t = T$  về 1). Tại bước thời gian  $t$ , nó tính toán trạng thái ẩn ngược  $\overleftarrow{\mathbf{h}}_t$  dựa trên đầu vào hiện tại  $\mathbf{x}_t$  và trạng thái ẩn từ "tương lai"  $\overleftarrow{\mathbf{h}}_{t+1}$ :

$$\overleftarrow{\mathbf{h}}_t = \text{LSTM}_{backward}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1})$$

## 2. Tích hợp thông tin và tính toán đầu ra

Tại mỗi bước thời gian  $t$ , đầu ra cuối cùng của mạng không chỉ phụ thuộc vào quá khứ mà còn dựa vào tương lai. Trạng thái ẩn xuôi  $\vec{\mathbf{h}}_t$  và trạng thái ẩn ngược  $\overleftarrow{\mathbf{h}}_t$  được kết hợp lại với nhau (thường là phép nối ghép véc-tơ - concatenation).

Trong Hình 2.15, sự kết hợp này được biểu diễn bằng các mũi tên từ hai khối LSTM cùng hướng về một nút kích hoạt  $\sigma$  (Sigmoid hoặc Softmax tùy bài toán) để tạo ra dự đoán đầu ra  $\mathbf{y}_t$ :

$$\mathbf{y}_t = \sigma(\mathbf{W}_y[\vec{\mathbf{h}}_t \oplus \overleftarrow{\mathbf{h}}_t] + \mathbf{b}_y)$$

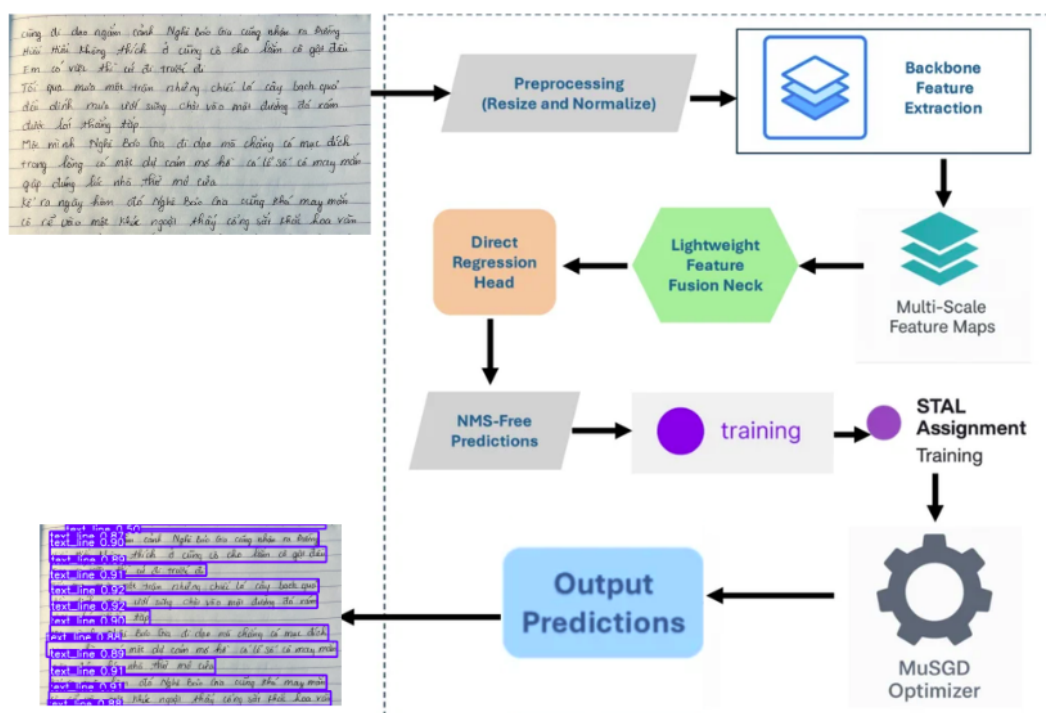
Trong đó:

- $\oplus$  đại diện cho phép toán nối (concatenate) hoặc cộng hai véc-tơ trạng thái ẩn.
- $\mathbf{W}_y$  và  $\mathbf{b}_y$  lần lượt là ma trận trọng số và độ lệch của lớp đầu ra.

Nhờ kiến trúc song song này, tại bất kỳ thời điểm  $t$  nào, mạng BiLSTM đều sở hữu lượng thông tin đầy đủ nhất về toàn bộ chuỗi dữ liệu (cả trước và sau  $\mathbf{x}_t$ ), từ đó đưa ra các dự đoán chính xác và mang tính ngữ cảnh toàn cục cao hơn so với LSTM đơn hướng.

## Chương 3 PHƯƠNG PHÁP

### 3.1 Module 1 : Phát hiện dòng chữ trong văn bản



Hình 3.1: Tổng quan quy trình phát hiện dòng chữ bằng YOLO26 [18]

Quy trình xử lý của mô hình bắt đầu bằng việc tiền xử lý ảnh đầu vào thông qua các bước thay đổi kích thước (Resize) và chuẩn hóa điểm ảnh (Normalize) để tối ưu tính toán. Tiếp theo, dữ liệu được đưa qua mạng cơ sở (Backbone) nhằm trích xuất các bản đồ đặc trưng đa kích thước (Multi-Scale Feature Maps). Điều này giúp hệ thống linh hoạt nhận diện các dòng chữ có độ lớn và khoảng cách khác nhau. Các đặc trưng này sau đó được tổng hợp hiệu quả qua khối cổ hợp nhất tinh gọn (Lightweight Feature Fusion Neck) mà không làm tổn nhiều tài nguyên tính toán[19].

Các đặc trưng sau khi dung hợp sẽ tiến thẳng vào đầu hồi quy (Direct Regression Head) để dự đoán trực tiếp tọa độ các dòng văn bản. Điểm đột phá của kiến trúc này

là xuất ra kết quả mà không cần đến bước hậu xử lý phức tạp (NMS-Free Predictions), giúp triệt tiêu độ trễ và tăng tốc độ xử lý thực tế lên mức tối đa [18] Cuối cùng (Output Predictions), hệ thống trả về bức ảnh với các dòng chữ đã được khoanh vùng chính xác, đi kèm điểm độ tin cậy.

Riêng trong quá trình đào tạo (Training), mô hình sử dụng chiến lược gán nhãn thông minh (STAL) để đo lường sai số giữa dự đoán và thực tế. Dựa vào đó, thuật toán tối ưu hóa MuSGD sẽ tự động điều chỉnh các trọng số, giúp mạng lưới liên tục khắc phục lỗi và tăng cường độ chính xác sau mỗi chu kỳ học [18].

### 3.1.1 Chuẩn bị dữ liệu

Tập dữ liệu gồm 660 ảnh văn bản tiếng Việt, được thu thập từ nhiều nguồn khác nhau nên mang tính đa dạng và gần với dữ liệu thực tế.

| Tập dữ liệu | Số ảnh | Số nhãn |
|-------------|--------|---------|
| Train       | 506    | 506     |
| Validation  | 102    | 102     |
| Test        | 52     | 52      |

Bảng 3.1: Thông kê tập dữ liệu.

Trước hết, mỗi ảnh thường chứa nhiều dòng văn bản, với độ dài và khoảng cách giữa các dòng không đồng đều, gây khó khăn cho việc tách dòng và nhận diện. Bên cạnh đó, dữ liệu có sự khác biệt về góc chụp và bố cục, nhiều ảnh bị nghiêng hoặc lệch phối cảnh, làm tăng độ phức tạp trong xử lý.

Ngoài ra, kích thước và chất lượng ảnh không đồng nhất, bao gồm ảnh độ phân giải thấp, bị mờ hoặc nhiễu, ảnh hưởng đến quá trình trích xuất đặc trưng. Đặc biệt, dữ liệu tiếng Việt chứa nhiều dấu thanh và dấu phụ, các thành phần này nhỏ và dễ bị mất hoặc dính vào ký tự chính.



### 3.1.2 Tiền xử lý dữ liệu

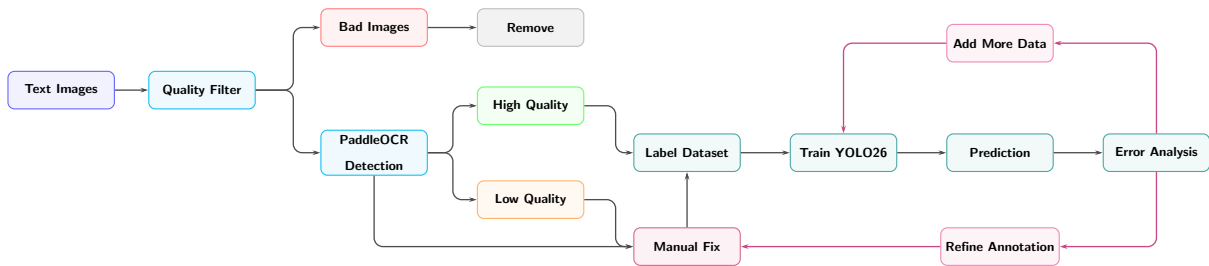
#### 1. Chiến thuật gán nhãn

Thực hiện gán nhãn theo từng dòng chữ (line-level labeling), trong đó mỗi dòng văn bản được coi là một đơn vị độc lập.

Phương pháp thực hiện bao gồm việc sử dụng các công cụ gán nhãn chuyên dụng (Labeling Tools) để xác định và vẽ các hộp giới hạn (bounding boxes) bao quanh chính xác từng dòng chữ trong hình ảnh, đảm bảo dữ liệu đầu vào phù hợp cho quá trình huấn luyện mô hình.

#### 2. Quy trình làm dữ liệu

Quy trình được thiết kế theo dạng vòng lặp khép kín, kết hợp giữa tự động hóa và kiểm soát thủ công để tối ưu hóa hiệu suất:



Hình 3.2: Hình ảnh quy trình xử lý dữ liệu

#### 1. Giai đoạn Sàng lọc

- *Text Images*: Tiếp nhận dữ liệu hình ảnh thô chứa văn bản.
- *Quality Filter*: Phân loại chất lượng ảnh đầu vào. Những ảnh không đạt yêu cầu (*Bad Images*) sẽ bị loại bỏ (*Remove*).

#### 2. Giai đoạn Gán nhãn tự động

- Sử dụng công cụ *PaddleOCR Detection* để tự động nhận diện và tạo các nhãn bao quanh dòng chữ cho các ảnh đạt chất lượng [20].

### 3. Giai đoạn Kiểm định và Chỉnh sửa

- *High Quality*: Những ảnh được máy gán nhãn chính xác sẽ được chuyển thẳng vào tập dữ liệu nhãn (*Label Dataset*).
- *Low Quality / Manual Fix*: Những trường hợp máy nhận diện sai hoặc thiếu sẽ được chuyển qua bước chỉnh sửa thủ công bởi con người để đảm bảo độ chính xác tuyệt đối trước khi đưa vào tập dữ liệu.

### 4. Giai đoạn Huấn luyện và Đánh giá

- *Train YOLO26*: Sử dụng tập dữ liệu đã gán nhãn chuẩn để huấn luyện mô hình YOLO26 [21].
- *Prediction*: Chạy mô hình trên dữ liệu thực tế để kiểm tra kết quả.
- *Error Analysis*: Phân tích các lỗi sai của mô hình để thực hiện các bước cải tiến:
  - *Refine Annotation*: Tinh chỉnh lại các nhãn bị sai lệch.
  - *Add More Data*: Bổ sung thêm dữ liệu mới để tăng cường khả năng học của mô hình.

#### 3.1.3 Hàm mất mát của YOLO26

Hàm mất mát cốt lõi của YOLO26 được thiết kế lại hoàn toàn cho kiến trúc End-to-End (loại bỏ hậu xử lý NMS và module DFL) [18]. Tổng hàm mất mát  $L_{total}$  là sự kết hợp có trọng số động giữa hàm phân loại ( $L_{cls}$ ) và hàm hồi quy hộp bao ( $L_{box}$ ):

$$L_{total}(t) = \lambda_{cls}(t) \cdot L_{cls} + \lambda_{box}(t) \cdot L_{box} \quad (3.1)$$

Trong đó,  $t$  là tiến trình huấn luyện (epoch hiện tại). Khác với các thế hệ trước, YOLO26 áp dụng cơ chế **ProgLoss (Progressive Loss Balancing)**: trọng số không cố

định mà thay đổi theo thời gian. Ở giai đoạn đầu, mạng ưu tiên học phân loại đặc trưng ( $\lambda_{cls}$  lớn), và về cuối quá trình, hệ thống tự động dồn trọng số để tinh chỉnh ranh giới tọa độ ( $\lambda_{box}$  lớn)[19].

### 1. Hàm phân loại với Nhãn mục tiêu mềm

Để tối ưu hóa khả năng nhận diện các vật thể nhỏ (*Small-Target-Aware*), YOLO26 không sử dụng nhãn cứng (đúng/sai tuyệt đối) mà dùng nhãn mềm  $y_i$  để chấm điểm chất lượng thực tế của từng khung dự đoán:

$$L_{cls} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot (1 - \hat{p}_i)^\gamma \log(\hat{p}_i) + (1 - y_i) \cdot \hat{p}_i^\gamma \log(1 - \hat{p}_i)] \quad (3.2)$$

- $N$ : Tổng số lượng mẫu dữ liệu.
- $\hat{p}_i$ : Xác suất dự đoán của mô hình.
- $y_i$ : Điểm số mục tiêu mềm do cơ chế **STAL (Soft Target Assignment Loss)** gán tự động ( $y_i \in [0, 1]$ ).
- $\gamma$ : Hệ số hội tụ giúp giảm nhiễu từ các mẫu nền (mẫu âm tính dễ).

### 2. Hàm hồi quy hộp bao trực tiếp

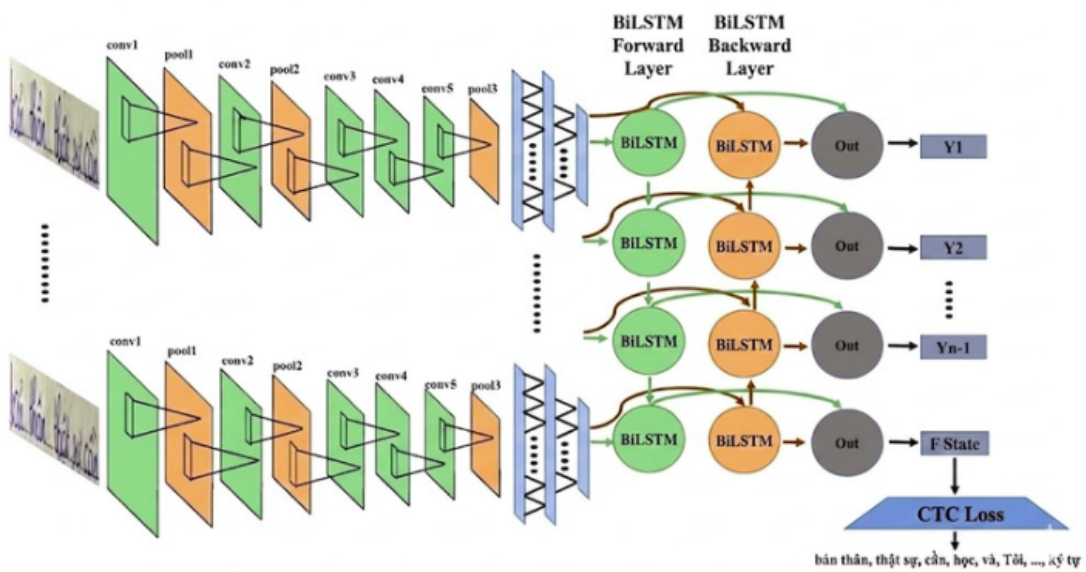
Bằng việc loại bỏ hoàn toàn module DFL (*Distribution Focal Loss*) nhằm giảm tải tính toán cho các thiết bị Edge CPU, việc tối ưu tọa độ trong YOLO26 phụ thuộc hoàn toàn vào **CIoU Loss (Complete IoU)**[22]. Hàm này ép mô hình phải học đồng thời 3 yếu tố: diện tích giao nhau, khoảng cách tâm và tỷ lệ khung hình:

$$L_{box} = 1 - IoU + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \quad (3.3)$$

- $IoU$ : Tỷ lệ phần diện tích giao nhau trên phần hợp nhất.

- $\rho^2(\mathbf{b}, \mathbf{b}^{gt})$ : Bình phương khoảng cách giữa hai tâm của hộp dự đoán và hộp chuẩn (Ground Truth).
- $c$ : Chiều dài đường chéo của hộp đóng kín nhỏ nhất chứa cả hai hộp.
- $\alpha v$ : Thành phần phạt độ lệch tỷ lệ khung hình (*Aspect Ratio*), ép hộp dự đoán phải ôm sát vật thể thực tế.

### 3.2 Module 2: Nhận diện văn bản chữ viết tay



Hình 3.3: Hình ảnh mạng CRNN

#### 1. Ảnh đầu vào

Hệ thống không xử lý trực tiếp toàn bộ ảnh lớn mà sử dụng các ảnh đã được cắt (crop) từ Module 1. Mỗi ảnh đầu vào chỉ chứa một dòng chữ viết tay duy nhất. Cách tiếp cận này giúp đơn giản hóa bài toán, cho phép mô hình tập trung vào việc nhận diện chuỗi ký tự theo thứ tự từ trái sang phải, đồng thời giảm nhiễu từ các vùng không liên quan.

## **2. Trích xuất đặc trưng**

Ảnh đầu vào được đưa qua các lớp tích chập (CNN) để trích xuất đặc trưng hình ảnh[5], [16]. CNN đóng vai trò như một bộ trích xuất đặc trưng, có khả năng phát hiện các thành phần cơ bản như nét dọc, nét ngang, đường cong và cấu trúc hình học của ký tự.

Đầu ra của bước này là một chuỗi đặc trưng (feature sequence), trong đó ảnh được biểu diễn dưới dạng các vector đặc trưng theo từng vị trí từ trái sang phải. Có thể hình dung quá trình này như việc chia ảnh thành nhiều lát cắt dọc, mỗi lát chứa thông tin của một phần chữ viết.

## **3. Học ngữ cảnh chuỗi**

Do chữ viết tay thường có hiện tượng dính ký tự và khó phân tách, mô hình sử dụng mạng BLSTM (Bidirectional Long Short-Term Memory) để học ngữ cảnh [5], [6].

BLSTM xử lý chuỗi đặc trưng theo hai chiều: từ trái sang phải và từ phải sang trái. Nhờ đó, mô hình có thể tận dụng thông tin ngữ cảnh ở cả hai phía để dự đoán chính xác ký tự, đặc biệt trong các trường hợp ký tự bị mờ hoặc không rõ ràng.

## **4. Phân loại và dự đoán ký tự**

Sau khi đã học được ngữ cảnh, dữ liệu được đưa qua các lớp Fully Connected để thực hiện phân loại. Lớp cuối cùng sử dụng hàm Softmax để tính toán xác suất của từng ký tự trong bảng chữ cái tiếng Việt tại mỗi vị trí trong chuỗi.

Kết quả đầu ra là một chuỗi các xác suất tương ứng với các ký tự dự đoán.

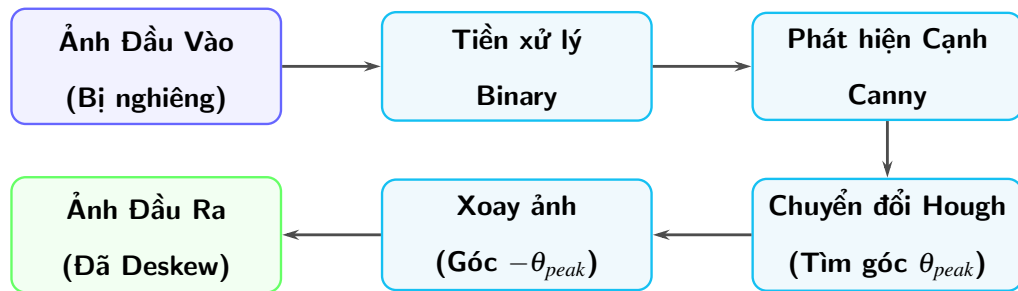
## **5. CTC**

Để chuyển đổi chuỗi dự đoán thành văn bản hoàn chỉnh, mô hình sử dụng thuật toán CTC Loss, cho phép ánh xạ chuỗi đầu ra có độ dài thay đổi thành chuỗi ký tự cuối cùng

bằng cách loại bỏ các ký tự lặp và ký tự rỗng (blank) [7].

### 3.2.1 Tiền xử lý ảnh

Khi scan hoặc chụp ảnh tài liệu, các dòng chữ thường không nằm ngang hoàn hảo mà sẽ bị nghiêng. Nếu đưa ảnh nghiêng trực tiếp vào mạng CRNN, các lát cắt dọc sẽ cắt chéo qua nét chữ, làm giảm độ chính xác nghiêm trọng. Quy trình Deskew (Chỉnh nghiêng) bằng Hough Transform được áp dụng để giải quyết vấn đề này.



Hình 3.4: Quá trình xử lý dòng chữ

**Công thức chuyển đổi Hough Transform:**

$$\rho = x \cos \theta + y \sin \theta \quad (3.4)$$

Trong đó, các đại lượng được định nghĩa cụ thể như sau:

- $(x, y)$ : Tọa độ của một điểm ảnh (pixel) bất kỳ trong không gian ảnh ban đầu (thường là các điểm thuộc nét chữ đã được trích xuất sau bước phát hiện cạnh Canny).
- $\rho$  (**Rho**): Khoảng cách vuông góc tính từ gốc tọa độ (thường là góc trên cùng bên trái của ảnh) đến đường thẳng đang xét.
- $\theta$  (**Theta**): Góc tạo bởi trục hoành ( $x$ ) và đoạn thẳng vuông góc nối từ gốc tọa độ đến đường thẳng đó. Giá trị  $\theta$  này đại diện cho độ nghiêng của dòng chữ.

## Nguyên lý hoạt động trong không gian tham số:

Thay vì sử dụng phương trình đường thẳng dạng  $y = ax + b$  (gặp khó khăn khi đường thẳng đứng có hệ số góc  $a$  tiến tới vô cùng), Hough Transform chuyển bài toán sang không gian tham số  $(\rho, \theta)$ .

- Với mỗi điểm  $(x, y)$  trong không gian ảnh, có vô số đường thẳng đi qua nó, tương ứng với một đường cong hình sin trong không gian  $(\rho, \theta)$ .
- Nếu nhiều điểm  $(x, y)$  cùng nằm trên một đường thẳng, các đường cong hình sin tương ứng của chúng sẽ giao nhau tại một điểm duy nhất  $(\rho_{peak}, \theta_{peak})$ .
- Điểm giao nhau này chính là "phiếu bầu" cao nhất, giúp thuật toán xác định được góc nghiêng chính xác của dòng chữ để thực hiện phép xoay (Deskew) về trạng thái nằm ngang ( $\theta = 0^\circ$ ).

## 1. Ảnh đầu vào

Hệ thống nhận một hình ảnh dòng chữ viết tay đã được crop, nhưng nó đang bị xô lệch (ví dụ góc nghiêng ban đầu  $\theta_{skew} \approx +5.8^\circ$ ).

## 2. Quy trình deskew

Quy trình này gồm 4 bước nối tiếp nhau để tìm và sửa lỗi nghiêng:

- **Tiền xử lý Binary (Binary Conversion):** Ảnh gốc (thường có màu xám hoặc nhiều) được chuyển hóa thành dạng nhị phân (chỉ gồm 2 màu đen và trắng). Việc này giúp tách bạch hoàn toàn phần nét chữ (foreground) ra khỏi phần giấy nền (background).
- **Phát hiện cạnh Canny (Canny Edge Detection):** Thay vì xử lý toàn bộ các điểm ảnh nét chữ, thuật toán Canny sẽ quét qua ảnh nhị phân và chỉ giữ lại các đường

viền (cạnh). Điều này làm giảm đáng kể khối lượng dữ liệu, chỉ giữ lại bộ khung hình học giúp phát hiện đường thẳng.

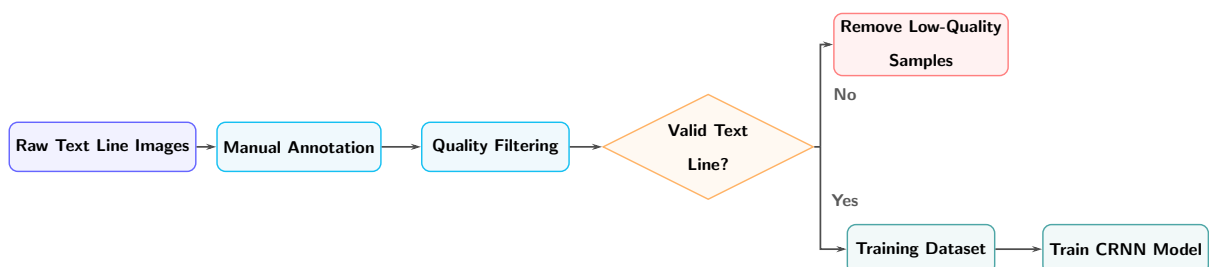
- **Chuyển đổi Hough (Hough Transform):** Đây là bước cốt lõi để tìm ra góc nghiêng của dòng chữ. Thuật toán xét từng điểm ảnh trên các đường cạnh Canny và ánh xạ chúng sang không gian Hough (dưới dạng các đường cong hình sin). Nơi các đường cong này giao nhau nhiều nhất (điểm  $\theta_{peak}$ ) chính là góc nghiêng thực tế của dòng chữ.
- **Xoay (Rotation):** Khi đã biết chính xác dòng chữ đang bị nghiêng ở góc  $\theta_{peak}$  (ví dụ  $+5.8^\circ$ ), hệ thống áp dụng phép xoay hình học để bẻ ngược bức ảnh lại một góc bù trừ ( $-\theta_{peak}$ ).

### 3. Đầu ra

Kết quả cuối cùng là một bức ảnh chứa dòng chữ đã được điều chỉnh cho nằm ngang hoàn hảo (góc  $0^\circ$ ). Bức ảnh ngay ngắn này giờ đây đã sẵn sàng để được đưa vào mạng CRNN để trích xuất đặc trưng và nhận diện.

#### 3.2.2 Quá trình làm dữ liệu

Quy trình chuẩn bị dữ liệu và huấn luyện mô hình được thực hiện qua các giai đoạn nghiêm ngặt nhằm đảm bảo độ chính xác tối ưu cho hệ thống nhận dạng. Các bước cụ thể được mô tả như sau:



Hình 3.5: Quá trình xử lý dữ liệu



- **Raw Text Line Images (Dữ liệu thô):** Tập hợp các hình ảnh dòng văn bản được cắt trích ra sau quá trình phát hiện dòng chữ của module 1.
- **Manual Annotation (Gán nhãn thủ công):** Sử dụng một tool label tự tạo để gán nhãn thủ công chuyển đổi thông tin từ dạng hình ảnh sang định dạng văn bản kỹ thuật số. Bước này tạo ra tập dữ liệu đối sánh chuẩn (*ground-truth*) phục vụ cho việc giám sát quá trình học của máy.
- **Quality Filtering (Lọc nhiễu và Kiểm soát chất lượng):** Sau khi gán nhãn, dữ liệu được đưa qua bộ lọc để đánh giá tính hợp lệ (*Valid Text Line?*). Quá trình này rẽ nhánh dựa trên chất lượng mẫu:
  - *Trường hợp không đạt (No):* Các mẫu có hình ảnh mờ nhòe hoặc nhãn sai lệch sẽ bị loại bỏ thông qua bước **Remove Low-Quality Samples**.
  - *Trường hợp đạt chuẩn (Yes):* Các dữ liệu tinh sạch sẽ được đưa vào **Training Dataset**.
- **Train CRNN Model (Huấn luyện mô hình):** Sử dụng tập dữ liệu đã qua sàng lọc để huấn luyện mạng nơ-ron tích chập tuần tự (*Convolutional Recurrent Neural Network - CRNN*), giúp mô hình học cách ánh xạ từ đặc trưng hình ảnh sang chuỗi ký tự văn bản [5].

### 3.2.3 Dataset

Để huấn luyện mô hình CRNN (Convolutional Recurrent Neural Network) cho bài toán nhận dạng chữ, tập dữ liệu đầu vào bao gồm các hình ảnh chứa các dòng văn bản đã được cắt ra (crop) từ giai đoạn phát hiện (detection) trước đó. Các hình ảnh này được thu thập với sự biến dạng về bối cảnh, góc chụp và điều kiện ánh sáng để tăng cường khả năng tổng quát hóa của mô hình. Toàn bộ dữ liệu được phân chia theo tỉ lệ: 80% cho tập huấn luyện (Train), 15% cho tập kiểm tra (Validation) và 5% cho tập đánh giá (Test).

| Tập dữ liệu | Số ảnh | Số nhãn |
|-------------|--------|---------|
| Train       | 1534   | 1534    |
| Validation  | 285    | 285     |
| Test        | 94     | 94      |

Bảng 3.2: Thống kê tập dữ liệu hình ảnh dòng chữ.

### 3.2.4 Hàm mất mát

Trong các bài toán nhận dạng chuỗi như nhận dạng chữ viết (OCR), sự giống hệt (alignment) giữa chuỗi đặc trưng đầu vào và chuỗi ký tự đầu ra thường không được biết trước do sự chênh lệch về độ dài. Để giải quyết triệt để vấn đề này, mô hình sử dụng hàm mất mát **CTC (Connectionist Temporal Classification)** [7].

- **Xác suất chuỗi trong CTC:**

Xác suất để mô hình dự đoán đúng chuỗi mục tiêu  $y$  khi cho trước đầu vào  $x$  được định nghĩa là tổng xác suất của tất cả các đường dẫn (paths) hợp lệ có thể sinh ra  $y$ :

$$P(y|x) = \sum_{\pi \in B^{-1}(y)} P(\pi|x)$$

Trong đó, do giả định các dự đoán tại mỗi thời điểm là độc lập, xác suất của một đường dẫn cụ thể  $\pi$  được tính bằng tích các xác suất dự đoán tại từng bước thời gian  $t$ :

$$P(\pi|x) = \prod_{t=1}^T p_{\pi_t}^{(t)}$$

**Giải thích các đại lượng:**

- $x$ : Chuỗi đặc trưng đầu vào (được trích xuất từ các lớp mạng nơ-ron trước đó).
- $y$ : Chuỗi nhãn thực tế (*ground-truth*) cần dự đoán.
- $\pi$ : Một đường dẫn dự đoán thô ở cấp độ từng khung hình (*frame*). Đường dẫn này có độ dài bằng  $T$ , bao gồm cả các ký tự lặp lại và ký hiệu rỗng (*blank*).

- $B$ : Hàm ánh xạ (thu gọn) làm nhiệm vụ loại bỏ các ký tự lặp lại liên kề và các ký hiệu rỗng từ đường dẫn thô  $\pi$  để tạo ra chuỗi đầu ra cuối cùng.
- $B^{-1}(y)$ : Tập hợp tất cả các đường dẫn  $\pi$  hợp lệ mà sau khi đi qua hàm ánh xạ  $B$  sẽ tạo thành chuỗi mục tiêu  $y$ .
- $T$ : Tổng số bước thời gian (*time steps*) của đặc trưng đầu vào.
- $p_{\pi_t}^{(t)}$ : Xác suất mà mạng nơ-ron dự đoán ra ký tự  $\pi_t$  tại thời điểm  $t$ .

• **Hàm mất mát CTC Loss:**

Mục tiêu cốt lõi của quá trình huấn luyện là cực đại hóa xác suất dự đoán đúng  $P(y|x)$ . Tương tự như hàm Cross-Entropy, để thuận tiện cho các thuật toán tối ưu hóa (như Gradient Descent), CTC Loss sử dụng hàm logarit âm (*Negative Log-Likelihood*):

$$\mathcal{L}_{CTC} = -\log P(y|x)$$

Việc cực tiểu hóa giá trị lỗi  $\mathcal{L}_{CTC}$  tương đương với việc cực đại hóa xác suất  $P(y|x)$ . Thông qua quá trình lan truyền ngược (*backpropagation*), mạng nơ-ron sẽ tự động điều chỉnh các trọng số để mô hình đưa ra dự đoán tiệm cận nhất với nhãn thực tế.

## Chương 4 Thực nghiệm và thảo luận

### 4.1 Kết quả thực nghiệm

Trong phần này, nghiên cứu tiến hành trình bày các thiết lập thực nghiệm và đánh giá hiệu quả của phương pháp đề xuất đối với bài toán nhận diện chữ viết tay tiếng Việt. Kết quả thực nghiệm được phân tích thông qua các chỉ số đánh giá và đối chiếu với một số kiến trúc mô hình khác nhằm làm rõ tính khả thi, độ chính xác và khả năng cải thiện hiệu năng của phương pháp nghiên cứu.

#### 4.1.1 Đánh giá mô hình

Sử dụng các chỉ số đánh giá gồm Character Accuracy (Char Acc), Character Error Rate (CER), Word Accuracy (Word Acc) và Sequence Accuracy (Seq Acc) nhằm đánh giá toàn diện hiệu năng của mô hình trong bài toán nhận diện chữ viết tay tiếng Việt. Kết quả thực nghiệm và so sánh giữa các kiến trúc backbone khác nhau khi kết hợp với mô hình CRNN được trình bày trong Bảng 4.1.

| Model                  | Char Acc (%) | CER (%)     | Word Acc (%) | Seq Acc (%) |
|------------------------|--------------|-------------|--------------|-------------|
| CRNN                   | 82.8         | 17.2        | 68.5         | 55.2        |
| CRNN + VGG16           | 85.4         | 14.6        | 74.2         | 58.7        |
| CRNN + VGG19           | 83.6         | 16.4        | 72.1         | 56.8        |
| CRNN + ResNet34        | 82.9         | 17.1        | 68.9         | 55.3        |
| <b>CRNN + ResNet50</b> | <b>88.8</b>  | <b>11.2</b> | <b>77.3</b>  | <b>61.5</b> |
| CRNN + EfficientNetB0  | 84.7         | 15.3        | 72.8         | 58.1        |

Bảng 4.1: Kết quả đánh giá hiệu năng của các mô hình trên tập dữ liệu chữ viết tay tiếng Việt.

#### 4.1.2 Phân tích và đánh giá kết quả

Kết quả thực nghiệm được trình bày trong Bảng 4.1, có thể khẳng định sự kết hợp giữa kiến trúc CRNN và mạng trích xuất đặc trưng (backbone) ResNet50 mang lại hiệu năng vượt trội nhất trong bài toán nhận diện chữ viết tay tiếng Việt [23]. Cụ thể, cấu hình CRNN + ResNet50 dẫn đầu ở toàn bộ các tiêu chí, đạt độ chính xác ký tự (Char Acc) cao nhất ở mức 88.8%, qua đó ép tỷ lệ lỗi ký tự (CER) xuống mức thấp nhất là 11.2%. Không chỉ xuất sắc ở cấp độ ký tự đơn lẻ, sự ưu việt của mô hình còn được thể hiện qua các độ đo khác hơn: độ chính xác mức từ (Word Acc) đạt 77.3% và độ chính xác toàn chuỗi (Seq Acc) đạt 61.5%. Sự nâng cấp toàn diện này so với mạng CRNN nguyên bản minh chứng cho việc ResNet50 đã trích xuất thành công các đặc trưng không gian phức tạp của chữ viết tay tiếng Việt – một ngôn ngữ vốn có độ biến thiên rất cao về phong cách viết, nét nổi và hệ thống dấu phụ (dấu thanh, dấu mũ).

Phân tích ở nhóm cấu hình tầm trung, các mô hình sử dụng backbone VGG16 và EfficientNetB0 ghi nhận những cải thiện tích cực so với mạng CRNN truyền thống [24], [25]. Điển hình, CRNN + VGG16 đạt Char Acc 85.4% (tương đương CER 14.6%), cho thấy năng lực biểu diễn đặc trưng hình thái khá ổn định. Dẫu vậy, khi đặt lên bàn cân với ResNet50, giới hạn của các mạng này nằm ở việc khó duy trì trọn vẹn thông tin không gian xuyên suốt các tầng tích chập sâu, dẫn đến hiệu suất giảm sút khi xử lý các chuỗi ký tự dính liền hoặc nét viết biến dạng.

Mặt khác, kết quả từ VGG19 và ResNet34 mang đến một góc nhìn thực nghiệm quan trọng: việc thay đổi độ sâu mạng không phải lúc nào cũng tỷ lệ thuận với sự gia tăng độ chính xác [23], [24]. Sự sụt giảm hiệu năng của VGG19 (CER 16.4%) so với VGG16 phản ánh khả năng mạng bị nhiễu thông tin hoặc rơi vào trạng thái quá khớp (overfitting) khi kiến trúc quá sâu nhưng thiếu cơ chế kết nối tắt phù hợp. Tương tự, khối cấu trúc cơ bản (Basic Block) của ResNet34 dường như chưa đủ năng lực để bóc tách các ký tự có hình dạng tương đồng hoặc thiếu nét, khiến kết quả nhận diện chỉ nhỉnh hơn biên độ hẹp

so với mô hình gốc [23].

Tóm lại, thực nghiệm đã khẳng định vai trò mang tính quyết định của việc lựa chọn mạng trích xuất đặc trưng trong hệ thống nhận diện chuỗi. Cơ chế học căn dư (residual learning) với khối Bottleneck của ResNet50 giúp mô hình mở rộng độ sâu hiệu quả mà không bị suy biến đạo hàm, dung hòa xuất sắc giữa khả năng học đặc trưng sâu và độ ổn định tổng thể. Do đó, CRNN kết hợp ResNet50 là cấu hình tối ưu và phù hợp nhất trong nghiên cứu này, tạo tiền đề vững chắc về độ tin cậy để triển khai các ứng dụng số hóa tài liệu chữ viết tay tiếng Việt trong thực tiễn [23].

## Chương 5 Tổng kết và hướng phát triển

### 5.1 Tổng kết

Nghiên cứu này đã đề xuất và xây dựng thành công phương pháp nhận diện chữ viết tay tiếng Việt dựa trên nền tảng kiến trúc mạng nơ-ron hồi quy tích chập (CRNN) [5]. Bằng việc kết hợp sức mạnh trích xuất đặc trưng không gian của mạng CNN và khả năng mô hình hóa chuỗi ngữ cảnh tuần tự của mạng RNN, hệ thống đã giải quyết triệt để những thách thức cốt lõi của bài toán OCR tiếng Việt, bao gồm sự đa dạng trong phong cách viết, biến thiên về độ nghiêng, độ nét, và đặc biệt là sự phức tạp của hệ thống dấu phụ (dấu thanh, dấu mũ) [5], [16].

Thông qua quá trình thực nghiệm đối sánh toàn diện giữa các mạng trích xuất đặc trưng (backbone) phổ biến như VGG16, VGG19, ResNet34, ResNet50 và Efficient-NetB0, nghiên cứu đã khẳng định kiến trúc CRNN kết hợp ResNet50 là giải pháp tối ưu nhất [23], [24], [25]. Mô hình này không chỉ dẫn đầu về độ chính xác ở cấp độ ký tự (giảm thiểu tối đa tỷ lệ lỗi CER) mà còn cải thiện vượt bậc khả năng nhận diện chuỗi văn bản hoàn chỉnh. Cơ chế học cặn dư (residual) của ResNet50 đã chứng minh sự ưu việt trong việc bóc tách và biểu diễn các đặc trưng hình thái phức tạp, giúp hệ thống duy trì tính ổn định, độ dung lỗi cao với nhiễu và khả năng tổng quát hóa ấn tượng trên nhiều mẫu chữ viết tay khác nhau.

### 5.2 Hạn chế và hướng phát triển

Dù đạt được những kết quả khả quan, nghiên cứu vẫn ghi nhận một số hạn chế nhất định. Hiện tại, mô hình vẫn bộc lộ điểm yếu khi xử lý các trường hợp chữ viết tay cực đoan (chữ quá xấu, viết tháu, dính liền nét) hoặc các ký tự bị biến dạng dẫn đến hình thái tương đồng nhau. Ngoài ra, quy mô và sự đa dạng của tập dữ liệu huấn luyện hiện

tại dù đáp ứng được yêu cầu cơ bản nhưng vẫn cần được đầu tư mở rộng để tối ưu hóa khả năng nhận diện của hệ thống trong môi trường thực tế.

Dựa trên những hạn chế còn tồn tại, định hướng phát triển tiếp theo của đề tài sẽ tập trung vào ba khía cạnh chính. Trước tiên, để tăng tính ổn định và khả năng ứng dụng thực tế, nghiên cứu dự kiến sẽ tiếp tục thu thập và làm phong phú thêm tập dữ liệu huấn luyện, đặc biệt bổ sung các mẫu chữ viết tay có độ khó cao, đa dạng nét chữ hoặc hình ảnh chụp trong điều kiện thực tế (giấy nhẵn, ánh sáng kém, mực nhòe). Thứ hai, việc tìm hiểu và tích hợp các mô hình ngôn ngữ tiếng Việt (Language Models) hoặc từ điển n-gram vào khâu hậu xử lý sẽ được ưu tiên nhằm giúp phương pháp tự động kiểm tra và sửa các lỗi chính tả dựa trên ngữ cảnh câu. Cuối cùng, em hướng tới việc thử nghiệm các kiến trúc mạng hiện đại hơn như mạng Transformer hoặc bổ sung cơ chế Attention thay cho RNN truyền thống, qua đó kỳ vọng giải quyết tốt hơn tình trạng nhận diện nhầm các ký tự có hình dáng giống nhau và xử lý hiệu quả hơn các chuỗi văn bản dài.



## Tài liệu tham khảo

- [1] Ray Smith. “An Overview of the Tesseract OCR Engine”. In: *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. IEEE, 2007, pp. 629–633. DOI: [10.1109/ICDAR.2007.4376991](https://doi.org/10.1109/ICDAR.2007.4376991). URL: <https://doi.org/10.1109/ICDAR.2007.4376991>.
- [2] S. Mori, C. Y. Suen, and K. Yamamoto. “Historical Review of OCR Research and Development”. In: *Proceedings of the IEEE* 80.7 (1992), pp. 1029–1058. DOI: [10.1109/5.156468](https://doi.org/10.1109/5.156468).
- [3] Baoguang Shi, Xiang Bai, and Cong Yao. “An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39.11 (2017), pp. 2298–2304. DOI: [10.1109/TPAMI.2016.2646371](https://doi.org/10.1109/TPAMI.2016.2646371). URL: <https://doi.org/10.1109/TPAMI.2016.2646371>.
- [4] Alex Graves and Jürgen Schmidhuber. “Offline Handwriting Recognition with Multidimensional Recurrent Neural Networks”. In: *Advances in Neural Information Processing Systems* 21 (2009). URL: <https://papers.nips.cc/paper/2008/hash/66368270ffd51418ec58bd793f2d9b1b>.
- [5] Baoguang Shi, Xiang Bai, and Cong Yao. “An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition”. In: *arXiv preprint arXiv:1507.05717* (July 2015). URL: <https://arxiv.org/abs/1507.05717>.
- [6] Mike Schuster and Kuldip K. Paliwal. “Bidirectional Recurrent Neural Networks”. In: *IEEE Transactions on Signal Processing* 45.11 (Nov. 1997), pp. 2673–2681. URL: <https://ieeexplore.ieee.org/document/650093>.
- [7] Alex Graves et al. “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks”. In: *Proceedings of the 23rd International Conference on Machine Learning* (June 2006), pp. 369–376. URL: [https://www.cs.toronto.edu/~graves/icml\\_2006.pdf](https://www.cs.toronto.edu/~graves/icml_2006.pdf).
- [8] Thomas Plötz and Horst Bunke. “A Survey of Online and Offline Handwritten Text Recognition”. In: *Proceedings of the 12th International Conference on Frontiers in Handwriting Recognition* (2010). URL: <https://doi.org/10.1109/ICFHR.2010.44>.
- [9] Ray Smith. “An Overview of the Tesseract OCR Engine”. In: *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)* (2007), pp. 629–633. URL: <https://ieeexplore.ieee.org/document/4376991>.

- [10] André G. B. Baird and N. E. Whiddett. “A Survey of Optical Character Recognition Techniques”. In: *Pattern Recognition* (1992), pp. 1–15.
- [11] Ray Smith. “An Overview of the Tesseract OCR Engine”. In: *ICDAR* (2007). URL: <https://ieeexplore.ieee.org/document/4376991>.
- [12] Minghao Li et al. “TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 37.11 (2023), pp. 13094–13102. DOI: [10.1609/aaai.v37i11.26452](https://doi.org/10.1609/aaai.v37i11.26452). URL: <https://doi.org/10.1609/aaai.v37i11.26452>.
- [13] Xiaohua Zhai et al. “Sigmoid Loss for Language Image Pre-Training”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2023), pp. 11975–11986. DOI: [10.1109/ICCV51070.2023.01100](https://doi.org/10.1109/ICCV51070.2023.01100). URL: <https://doi.org/10.1109/ICCV51070.2023.01100>.
- [14] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [15] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016, pp. 779–788. DOI: [10.1109/CVPR.2016.91](https://arxiv.org/abs/1506.02640). URL: <https://arxiv.org/abs/1506.02640>.
- [16] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep Learning”. In: *Nature* 521.7553 (2015), pp. 436–444. DOI: [10.1038/nature14539](https://arxiv.org/abs/1404.7828). URL: <https://arxiv.org/abs/1404.7828>.
- [17] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. “Speech Recognition with Deep Recurrent Neural Networks”. In: *arXiv preprint arXiv:1303.5778* (2013). URL: <https://arxiv.org/abs/1303.5778>.
- [18] Ranjan Sapkota et al. “YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection”. In: (2025). arXiv: [2509.25164](https://arxiv.org/abs/2509.25164) [cs.CV]. URL: <https://arxiv.org/abs/2509.25164>.
- [19] Priyanto Hidayatullah and Refdinal Tubagus. “YOLO26: A Comprehensive Architecture Overview and Key Improvements”. In: *arXiv preprint arXiv:2602.14582* (2026). URL: <https://arxiv.org/abs/2602.14582>.
- [20] Yuning Du et al. “PP-OCR: A Practical Ultra Lightweight OCR System”. In: *arXiv preprint arXiv:2009.09941* (2020). URL: <https://arxiv.org/abs/2009.09941>.
- [21] Glenn Jocher, Jing Qiu, and Ayush Chaurasia. *Ultralytics YOLO26*. 2026. URL: <https://github.com/ultralytics/ultralytics>.
- [22] Zhaohui Zheng et al. “Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression”. In: *arXiv preprint arXiv:1911.08287* (2019). URL: <https://arxiv.org/abs/1911.08287>.

- [23] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90). URL: <https://arxiv.org/abs/1512.03385>.
- [24] Karen Simonyan and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *arXiv preprint arXiv:1409.1556* (2014). URL: <https://arxiv.org/abs/1409.1556>.
- [25] Mingxing Tan and Quoc V. Le. “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, pp. 6105–6114. URL: <https://arxiv.org/abs/1905.11946>.

## Phụ lục A Các chỉ số đánh giá

Việc đánh giá hiệu năng của phương pháp nghiên cứu được thực hiện qua hai giai đoạn độc lập: phát hiện vùng chứa văn bản (Detection) và nhận dạng nội dung ký tự (Recognition).

### I. Đánh giá mô hình Phát hiện đối tượng

Trong bài toán này, hiệu suất mô hình được xác định dựa trên sự tương quan về vị trí không gian giữa hộp dự đoán và hộp thực tế.

#### 1. Chỉ số IoU

Chỉ số này đo lường mức độ trùng khớp về mặt không gian giữa hai vùng ảnh.

$$IoU = \frac{Area(B_{pred} \cap B_{gt})}{Area(B_{pred} \cup B_{gt})} \quad (A.1)$$

**Trong đó :**

- $B_{pred}$ : Hình hộp giới hạn (*Bounding Box*) do mô hình dự đoán.
- $B_{gt}$ : Hình hộp giới hạn thực tế (*Ground Truth*) do người gán nhãn.
- $Area(B_{pred} \cap B_{gt})$ : Diện tích vùng giao nhau giữa hộp dự đoán và hộp thực tế.
- $Area(B_{pred} \cup B_{gt})$ : Tổng diện tích bao phủ của cả hai hộp giới hạn.

#### 2. Precision

Đo lường tỉ lệ dự đoán đúng trong số tất cả các dự đoán mà mô hình đưa ra.

$$Precision = \frac{TP}{TP + FP} \quad (A.2)$$

**Trong đó :**

- *TP (True Positive)*: Số lượng đối tượng dự đoán đúng.
- *FP (False Positive)*: Số lượng đối tượng dự đoán sai (nhận nhầm bối cảnh là đối tượng).

### **3. Recall**

Đo lường tỉ lệ đối tượng được tìm thấy trên tổng số đối tượng thực tế tồn tại.

$$Recall = \frac{TP}{TP + FN} \quad (A.3)$$

**Trong đó :**

- *TP (True Positive)*: Số lượng đối tượng dự đoán đúng.
- *FN (False Negative)*: Số lượng đối tượng thực tế bị mô hình bỏ sót.

### **4. Average Precision**

Đại diện cho hiệu năng của mô hình trên một lớp đối tượng cụ thể thông qua đường cong Precision-Recall.

$$AP = \int_0^1 P(R) dR \quad (A.4)$$

**Trong đó :**

- *R*: Chỉ số Recall.
- *P(R)*: Giá trị Precision tương ứng với từng mức độ Recall trên đồ thị.

## 5. Mean Average Precision

Giá trị trung bình của AP trên tất cả các lớp đối tượng cần nhận diện.

$$mAP = \frac{1}{C} \sum_{c=1}^C AP_c \quad (A.5)$$

**Trong đó :**

- $C$ : Tổng số lớp (classes) đối tượng trong tập dữ liệu.
- $AP_c$ : Giá trị Average Precision của lớp thứ  $c$ .

## II. Đánh giá mô hình Nhận dạng văn bản

Giai đoạn này sử dụng mô hình CRNN để chuyển đổi hình ảnh dòng chữ thành chuỗi văn bản tương ứng.

### 1. Độ chính xác ký tự

$$CharAcc = \frac{N_{char\_correct}}{N_{char\_total}} \times 100\% \quad (A.6)$$

**Trong đó :**

- $N_{char\_correct}$ : Số lượng ký tự được mô hình nhận dạng đúng.
- $N_{char\_total}$ : Tổng số lượng ký tự có trong nhãn thực tế.

### 2. Độ chính xác từ

$$WordAcc = \frac{N_{word\_correct}}{N_{word\_total}} \times 100\% \quad (A.7)$$

**Trong đó :**

- $N_{word\_correct}$ : Số lượng từ được nhận dạng đúng hoàn toàn (tất cả ký tự trong từ phải đúng).
- $N_{word\_total}$ : Tổng số từ có trong tập dữ liệu kiểm tra.

### 3. Độ chính xác chuỗi

$$SeqAcc = \frac{N_{seq\_correct}}{N_{seq\_total}} \times 100\% \quad (A.8)$$

**Trong đó :**

- $N_{seq\_correct}$ : Số lượng dòng văn bản mô hình nhận dạng đúng tuyệt đối 100%.
- $N_{seq\_total}$ : Tổng số dòng văn bản có trong tập dữ liệu kiểm tra.

### 4. Tỷ lệ lỗi ký tự

Đánh giá mức độ sai lệch dựa trên khoảng cách Levenshtein giữa chuỗi dự đoán và chuỗi thực tế.

$$CER = \frac{S + D + I}{N} \times 100\% \quad (A.9)$$

**Trong đó :**

- $S$  (*Substitution*): Số ký tự bị thay thế sai (nhận diện nhầm).
- $D$  (*Deletion*): Số ký tự bị thiếu hụt (bỏ sót).
- $I$  (*Insertion*): Số ký tự bị dư thừa so với nhãn gốc.
- $N$ : Tổng số ký tự có trong nhãn gốc (*Ground Truth*).